Intelligent Agents for Software Engineering: A Systematic Literature Review

HAOYI ZHANG, Xi'an Jiaotong-Liverpool University, China
HUAIJIN RAN, Xi'an Jiaotong-Liverpool University, China
XUNZHU TANG*, University of Luxembourg, Luxembourg

Following the advent of large language models (LLMs), Intelligent Agents have gained traction as effective solutions for addressing a wide range of tasks, particularly within software engineering (SE). However, understanding their definitions, architectures, and applications in SE remains nascent. To address this gap, we conducted a systematic literature review (SLR) of 215 peer-reviewed articles published between April 2023 and July 2025, synthesizing evidence on how Intelligent Agents are conceived, classified, and evaluated across SE tasks. Following a rigorous search strategy, study selection, and data-extraction protocol, we developed a comprehensive classification framework that captures both agent typologies and task-specific deployments. We answer three research questions: (1) How are Intelligent Agents defined and applied to particular SE tasks? (2) What core agent-based techniques underpin each task category? (3) What challenges and research avenues emerge when embedding Intelligent Agents in SE practice? Our analysis reveals trends in multi-agent collaboration, retrieval-augmented pipelines, and feedback-driven refinement, highlights gaps in evaluation metrics and tooling support, and proposes a research agenda to guide future innovation. Building on these findings, we present a view of the state of the art, spotlight challenges, and outline promising directions for future investigation. All review artifacts are publicly available at https://github.com/Haoyi-Zhang/Agent4SE_SLR.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Software and its engineering** → **Software development techniques**; • **Computing methodologies** → **Artificial intelligence**.

Additional Key Words and Phrases: Software Engineering, Intelligent Agents, Systematic Literature Review

ACM Reference Format:
Haoyi Zhang, Huaijin Ran, and Xunzhu Tang. 2025. Intelligent Agents for Software Engineering: A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* X, Y, Article 1 (August 2025), 54 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Intelligent Agents have rapidly moved from concept to practice across software engineering (SE), catalyzed by advances in large language models (LLMs), multi-agent coordination, and tool-augmented automation. Beyond single-turn assistance, contemporary agents perceive heterogeneous SE artefacts, reason over long horizons, act on development environments, and refine behaviour from feedback. This shift promises end-to-end support across the software development

*Xunzhu Tang is the corresponding author (xunzhu.tang@uni.lu).

Authors' addresses: Haoyi Zhang, hyeliozhang@gmail.com, Xi'an Jiaotong-Liverpool University, SuZhou, China; Huaijin Ran, seventeen17510@gmail.com, Xi'an Jiaotong-Liverpool University, SuZhou, China; Xunzhu Tang, xunzhu.tang@uni.lu, University of Luxembourg, Luxembourg, Luxembourg.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Association for Computing Machinery.
1049-331X/2025/8-ART1 $15.00
https://doi.org/XXXXXXX.XXXXXXX

lifecycle (SDLC)—from early requirements and design to coding, testing, deployment, and evolution—while raising urgent questions about definitions, architectures, evaluation fidelity, safety, and integration at scale.

Motivation and gap. Despite intense activity, the landscape remains fragmented along three axes. *Conceptually*, the term "agent" spans from chat-style copilots to autonomous, tool-operating systems, with boundaries that are often implicit or inconsistent. *Technically*, implementations reuse similar building blocks—planning/orchestration, knowledge grounding over repositories, and deep tool integration—yet descriptions and interfaces are non-uniform, hindering replication and transfer. *Empirically*, reported results rely on heterogeneous tasks and short horizons, with limited attention to multi-agent coordination effects, repository-scale validation, human–AI interaction (HAI) quality, and transparent cSEost/latency accounting. A field-wide synthesis tailored to SE is therefore needed to consolidate concepts, connect technologies to tasks, and surface reliable evidence and gaps.

Scope and method in brief. We conduct a systematic literature review of **215** peer-reviewed studies published between **April 2023** and **July 2025**, selected via a multi-database search over five major digital libraries and a transparent, PRISMA-style screening pipeline with dual-review quality control. The search retrieved an initial corpus of **400** records; two-phase screening and full-text assessment yielded the final set of 215 studies with strong inter-rater reliability. The SLR is complemented by a standardized data-extraction protocol and a task-driven, two-level classification that organizes all studies across the SE lifecycle. Public artefacts—including the screening protocol and classification outputs—are released to support reuse and longitudinal extension.

Research questions. Our investigation is organized around three interlocking questions that bridge concept, technology, and trajectory:

- **RQ1 (Concept & scope).** How are Intelligent Agents conceptualized in SE, and where are they currently applied across the lifecycle?
- **RQ2 (Technologies & patterns).** Which enabling architectures, planning/coordination strategies, knowledge-grounding pipelines, and tool integrations underpin these applications?
- **RQ3 (Challenges & directions).** What system-level barriers limit effectiveness, scalability, and trustworthiness in practice, and what research agenda can address them?

Synthesis at a glance. The classification uncovers ten Level-1 activity families spanning SDLC management, multi-agent collaboration, code analysis and quality assurance, automated testing, domain/industrial applications, education and research tooling, process automation, HAI collaboration, system integration and tool development, and legacy modernization/documentation. Across these families, agent implementations converge on four universal building blocks: (*i*) orchestration engines for long-horizon plans and multi-role workflows; (*ii*) retrieval-augmented grounding over repositories and project artefacts; (*iii*) transparent planning (e.g., chain/tree-of-thought) for auditable decisions; and (*iv*) deep tool integration with compilers, test frameworks, CI/CD, and observability stacks. Reported empirical benefits include noteworthy accuracy and productivity gains, alongside advances in quality-assurance automation; at the same time, results reveal persistent deficits in system-level evaluation, cost/latency reporting, and failure taxonomy coverage.

Contributions. This article makes the following contributions:

(1) **A clarified operational definition and typology.** We reconcile classic agent properties with modern LLM-centric pipelines, delineating essential traits (goal-directed autonomy, context awareness, planning and reasoning, continual learning, and social interaction) and recurring designs from single reactive agents to hierarchical and feedback-driven multi-agent systems.

(2) **A task-driven map of SE applications.** We systematize how agents are instantiated across the ten activity families, connecting role schemas and collaboration patterns to concrete tasks and evaluation settings, and quantifying where research is concentrated versus under-explored.
(3) **A unifying technological framework.** We distill four cross-cutting pillars—*orchestration engines*, *retrieval-augmented grounding*, *transparent planners* (CoT/ToT), and *tool adapters/integration*—and analyze trade-offs, dependencies, and failure modes across tasks and scales.
(4) **A consolidated evidence base.** We synthesize empirical effects (e.g., accuracy/productivity deltas and QA automation dividends) and identify systematic reporting gaps (e.g., trajectory logs, memory footprints, and end-to-end cost), establishing priorities for more faithful, reproducible evaluation.
(5) **A research agenda for robust, accountable adoption.** We articulate six system-level challenge categories—evaluation and benchmarks; scalability and efficiency; safety and trustworthiness; HAI and mixed-initiative control; integration and deployment; and privacy/governance—and propose actionable directions to align advances across 2025–2027.

Implications and roadmap. Taken together, our findings position Intelligent Agents as a unifying paradigm for SE automation, provided that evaluation fidelity, efficiency, safety, and integration debt are addressed in concert. The remainder of the article details the SLR methodology and classification, synthesizes the conceptual and technological landscapes (RQ1–RQ2), and develops a challenges-to-directions roadmap for maturing agentic SE (RQ3), concluding with implications for research and practice.

2 RESEARCH APPROACH

This investigation employs a systematic literature review methodology to provide a comprehensive, unbiased synthesis of the current state of knowledge regarding Intelligent Agents in software engineering. The SLR approach was selected due to its rigorous, transparent, and reproducible nature, enabling systematic identification, evaluation, and synthesis of relevant research evidence while minimizing researcher bias and ensuring comprehensive coverage [73].

The rapidly evolving landscape of Intelligent Agents in software engineering, characterized by LLMs, multi-agent systems, and novel agent-based frameworks, necessitates a systematic approach that captures both foundational concepts and cutting-edge developments. Our methodology addresses the inherent challenges of reviewing a dynamic field where technological advances and practical applications are continuously evolving.

The systematic review process was structured around four interconnected methodological pillars: **Rigorous Search Strategy:** We developed a comprehensive search methodology encompassing multiple academic databases, strategic keyword formulation, and Boolean logic optimization to ensure maximum coverage while maintaining precision and relevance. **Systematic Study Selection Process:** A transparent, multi-stage filtering process implemented clearly defined inclusion and exclusion criteria, multiple reviewer validation, and systematic quality assessment to ensure only methodologically sound and directly relevant studies were included.
Structured Data Extraction and Synthesis: We implemented a standardized data extraction framework that systematically captured key study characteristics, methodological approaches, empirical findings, and theoretical contributions, employing both quantitative aggregation and qualitative thematic analysis.
Research Question-Driven Analysis Framework: The investigation was organized around three complementary research questions that systematically address the conceptual foundations, technological implementations, and future evolution of Intelligent Agents in software engineering.

Task-Driven Classification Framework: We developed a comprehensive two-level classification system that systematically organizes all surveyed works according to their primary software engineering activities and technical approaches, enabling both descriptive and quantitative analysis of research patterns and gaps.

Each methodological component was designed to work synergistically, creating a robust analytical framework that addresses the complexity and multidisciplinary nature of Intelligent Agents research while maintaining systematic rigor required for high-quality literature synthesis.

2.1 Search Strategy

The search strategy was meticulously designed to comprehensively capture relevant studies in the rapidly evolving field of Intelligent Agents in software engineering. Our approach employed a multi-faceted search methodology combining systematic database querying, strategic keyword selection, and rigorous filtering criteria, as illustrated in Figure 1.

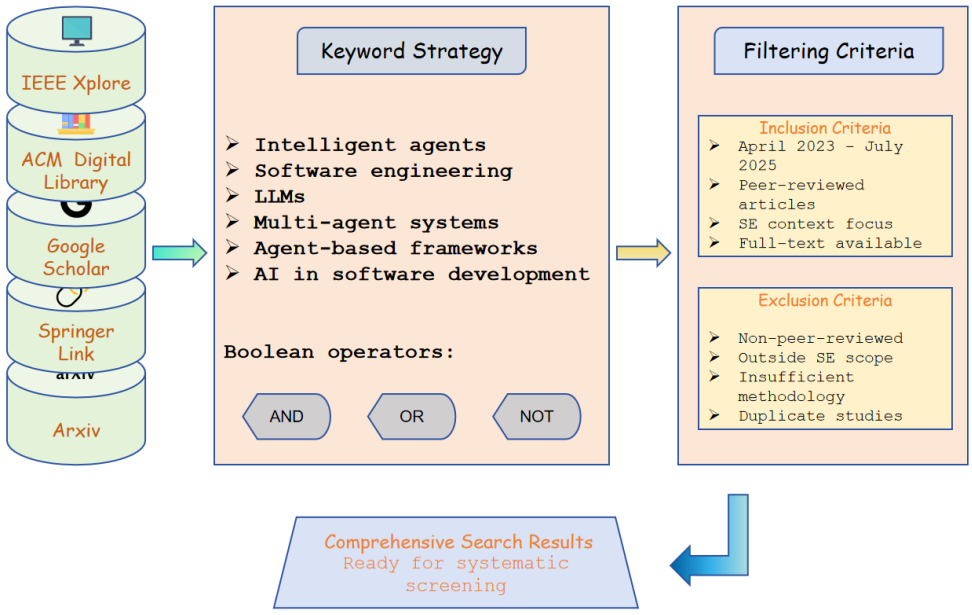


Fig. 1. Comprehensive Search Strategy Flow Diagram. The workflow demonstrates (1) systematic multi-database search across five major academic repositories, (2) strategic keyword combination using Boolean logic operators, and (3) rigorous filtering criteria application to ensure high-quality literature selection.

Database Selection and Coverage. Our comprehensive search strategy encompassed five major digital libraries, each selected for their distinct strengths to ensure maximum literature capture:

- **IEEE Xplore**: Premier repository for software engineering research, encompassing high-impact journals, conference proceedings, and technical standards with particular strength in systems and software engineering domains.
- **ACM Digital Library**: Comprehensive source for computing and software engineering literature, providing extensive coverage of theoretical foundations and practical applications.

- **Google Scholar**: Broad, multidisciplinary search engine capturing diverse publication types including academic articles, theses, and conference papers, ensuring coverage of emerging work.
- **SpringerLink**: Central platform for research in computer science, artificial intelligence, and software engineering, offering access to established and cutting-edge research.
- **arXiv**: Preprint archive providing early access to cutting-edge research in AI and software engineering, ensuring inclusion of the most recent developments.

Strategic Keyword Development. The keyword strategy balanced comprehensiveness with precision, employing a carefully curated combination of controlled vocabulary and free-text terms. The core search terms included:

- *"Intelligent Agents"* – capturing autonomous software entities
- *"Software engineering"* – ensuring domain specificity
- *"Large language models (LLMs)"* – addressing recent AI advancements
- *"Multi-agent systems"* – encompassing collaborative agent architectures
- *"Agent-based frameworks"* – targeting implementation approaches
- *"Artificial intelligence in software development"* – broadening AI application scope

These terms were systematically combined using Boolean operators (AND, OR, NOT) to construct precise search queries that maximized recall while maintaining high precision, ensuring retrieved articles addressed the intersection of Intelligent Agents and software engineering domains.

Inclusion and Exclusion Framework. We established stringent eligibility criteria consistently applied across all retrieved articles:

Inclusion Criteria:

- *Temporal relevance*: Publications dated April 2023–July 2025
- *Domain alignment*: Studies explicitly addressing Intelligent Agents within software engineering contexts
- *Publication quality*: Peer-reviewed journal articles, conference papers, and high-quality workshop proceedings
- *Accessibility*: Full-text availability for comprehensive analysis

Exclusion Criteria:

- *Quality assurance*: Non-peer-reviewed works or studies with insufficient empirical validation
- *Scope limitation*: Studies not primarily focusing on Intelligent Agents within software engineering contexts
- *Methodological adequacy*: Articles with insufficient detail for replication or lacking systematic evaluation approaches
- *Uniqueness*: Duplicate studies or overlapping publications to prevent bias

2.2 Study Selection Process

The study selection process systematically transformed the initial corpus into a refined collection of high-quality, directly relevant studies. Following established PRISMA guidelines [111], our methodology employed a transparent, multi-stage filtering process ensuring both comprehensive coverage and rigorous quality control. Figure 2 illustrates the complete selection workflow.

Initial Corpus Formation. The systematic search strategy yielded an initial corpus of 400 potentially relevant articles across five targeted academic databases. This collection represented comprehensive capture of literature addressing Intelligent Agents in software engineering contexts, encompassing diverse publication types, research methodologies, and application domains.

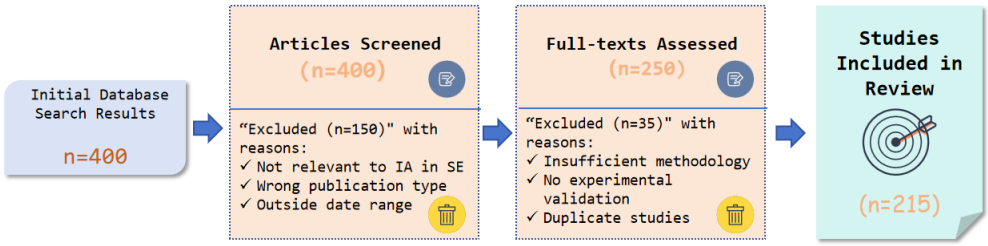


Fig. 2. PRISMA-Style Study Selection Workflow. The systematic filtering process shows progression from initial database search results (n=400) through title/abstract screening and full-text assessment to final included studies (n=215), with detailed exclusion criteria and reasons at each stage.

Two-Phase Selection Methodology. The study selection employed a systematic two-phase filtering process designed to efficiently identify relevant studies while maintaining comprehensive evaluation standards.

Phase 1: Title and Abstract Screening The initial screening phase focused on rapid identification of potentially relevant studies through systematic evaluation of titles and abstracts. This phase processed all 400 retrieved articles against inclusion criteria, with two independent reviewers conducting parallel screenings. Disagreements were resolved through discussion and, when necessary, consultation with a third reviewer.

The screening process excluded 150 articles that clearly fell outside the review scope: studies focused on non-software engineering domains (n=85), articles addressing general AI applications without specific agent focus (n=35), and publications lacking clear software engineering connections (n=30).

Phase 2: Full-Text Assessment and Quality Evaluation The second phase involved comprehensive evaluation of the remaining 250 articles through detailed full-text review, applying complete inclusion and exclusion criteria while conducting thorough quality assessment of research methodologies, empirical validation approaches, and contribution significance.

Each article underwent systematic evaluation across multiple dimensions: methodological rigor, empirical evidence quality, theoretical contribution clarity, and practical relevance to software engineering. Two independent reviewers conducted parallel assessments with systematic reconciliation procedures for resolving disagreements.

This evaluation phase resulted in exclusion of 35 additional articles due to insufficient methodological detail (n=15), lack of empirical validation (n=12), or limited relevance to Intelligent Agents in software engineering contexts (n=8).

Final Corpus Characteristics. The systematic selection process yielded a final corpus of 215 high-quality studies directly addressing Intelligent Agents in software engineering contexts, representing 53.75% of the initial retrieval. Quality assurance measures included inter-rater reliability assessment (κ = 0.87 for initial screening, κ = 0.82 for full-text review), systematic documentation of exclusion reasons, and audit trails for all selection decisions.

2.3 Data Extraction and Synthesis

The data extraction and synthesis process constitutes the analytical core of this systematic literature review, systematically transforming the curated corpus of 215 selected studies into structured knowledge through rigorous methodological procedures. This phase employed a standardized,

multi-dimensional approach designed to ensure comprehensive data capture while maintaining analytical objectivity [28]. Figure 3 demonstrates our comprehensive approach to knowledge extraction and structured synthesis.

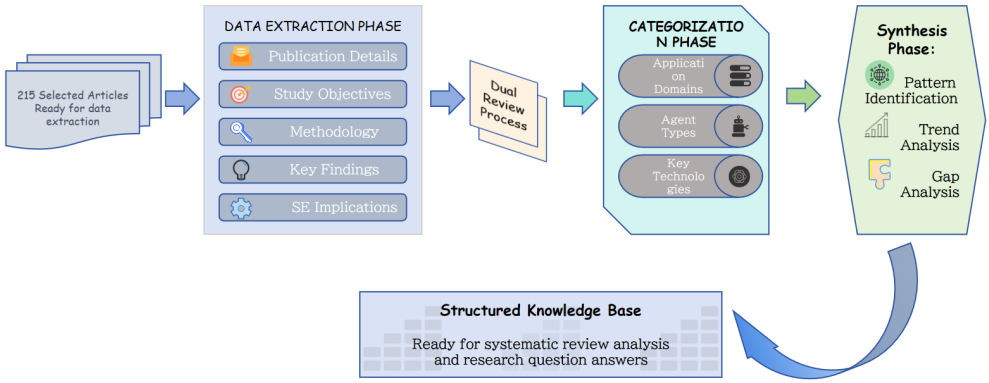


Fig. 3. Data Processing and Synthesis Workflow. The systematic five-phase process transforms 215 selected articles through standardized data extraction, dual review quality control, multi-dimensional categorization, and comprehensive synthesis to produce a structured knowledge base.

Standardized Data Extraction Framework. We developed a comprehensive extraction instrument capturing multi-dimensional information across five systematically defined domains:

- **Bibliometric and Publication Characteristics**: Complete bibliographic information including publication venue classification, temporal distribution, and author institutional affiliations
- **Research Focus and Scope Characterization**: Detailed documentation of study objectives, theoretical frameworks, and specific domains of Intelligent Agent application
- **Methodological Approach Documentation**: Comprehensive capture of research methodologies, experimental designs, validation approaches, and evaluation frameworks
- **Empirical Evidence and Contribution Analysis**: Systematic extraction of quantitative results, qualitative findings, performance metrics, and novel contributions
- **Contextual Application and Impact Assessment**: Analysis of practical applicability, implementation contexts, deployment challenges, and potential impacts on software engineering practice

Each article underwent independent parallel review by two trained researchers using the standardized protocol, achieving high inter-rater reliability (Cohen's $\kappa = 0.89$) while maintaining comprehensive data capture.

Multi-Dimensional Categorization. Following standardized extraction, we implemented comprehensive categorization across three complementary analytical dimensions:

Application Domain Taxonomy: Studies were systematically classified according to software engineering lifecycle phases and functional activities, revealing research concentration patterns and under-explored application areas.

Agent Architecture Classification: The categorization distinguished between architectural approaches including autonomous versus reactive behavioral models, single-agent versus multi-agent organizational structures, and traditional AI versus large language model technological foundations.

Technology and Technique Inventory: Studies underwent systematic analysis to identify underlying technologies, algorithmic approaches, and implementation techniques, encompassing machine learning methodologies, natural language processing techniques, and emerging technologies.

Systematic Synthesis and Pattern Analysis. The synthesis phase employed both quantitative aggregation techniques and qualitative thematic analysis to identify systematic patterns, evolutionary trends, and critical knowledge gaps. This approach combined statistical pattern recognition with interpretive analysis to generate robust insights into current capabilities and future development trajectories.To enable systematic analysis and comparison across the diverse landscape of Intelligent Agent applications, we complemented the data extraction process with a comprehensive classification framework that categorizes all 215 studies according to their primary software engineering focus and technical approaches.

2.4 Classification Framework

As a critical component of our systematic methodology, we developed a two-level, task-driven classification framework to address the fragmented nature of existing research and enable comprehensive analysis across the full software development lifecycle. This framework serves as both an organizational tool for consistent study annotation and an analytical instrument for identifying research hotspots, gaps, and emerging trends.

2.4.1 Overview of Categories. At Level 1, we partition intelligent-agent applications into ten broad software-engineering activities:

1. **Software Development Lifecycle Management (SDLC)** - Agents supporting core phases from requirements through maintenance
2. **Multi-Agent Systems and Collaboration** - Frameworks and protocols for coordinating multiple agents
3. **Code Analysis and Quality Assurance** - Tools for static analysis, code review, and vulnerability detection
4. **Automated Testing and Validation** - Agents that generate, execute, and evaluate test cases
5. **Industrial Applications and Domain-Specific Solutions** - Custom agents for manufacturing, cloud management, and specialized domains
6. **Educational and Research Applications** - Platforms for teaching, literature analysis, and benchmarking
7. **Development Process Automation** - Tools for DevOps, continuous integration, and project planning
8. **Human–AI Interaction and Collaboration** - Conversational interfaces and human-in-the-loop systems
9. **System Integration and Tool Development** - Agents for API orchestration and toolchain automation
10. **Legacy System Modernization and Documentation** - Techniques for analyzing and migrating legacy systems

Each Level 1 category is further decomposed into Level 2 subcategories capturing distinct technical approaches. For example, *Software Development Lifecycle Management* includes: Requirements Engineering and Analysis, Software Design and Architecture, Code Generation and Implementation, Testing and Quality Assurance, and Maintenance and Evolution.

We reviewed each of the 215 collected papers and assigned one Level 1 and one Level 2 label based on the paper's stated objectives, methods, and evaluation context. In cases of ambiguity, labels were discussed among the authors until consensus was reached. The complete classification is illustrated in Figures 4 and 5.

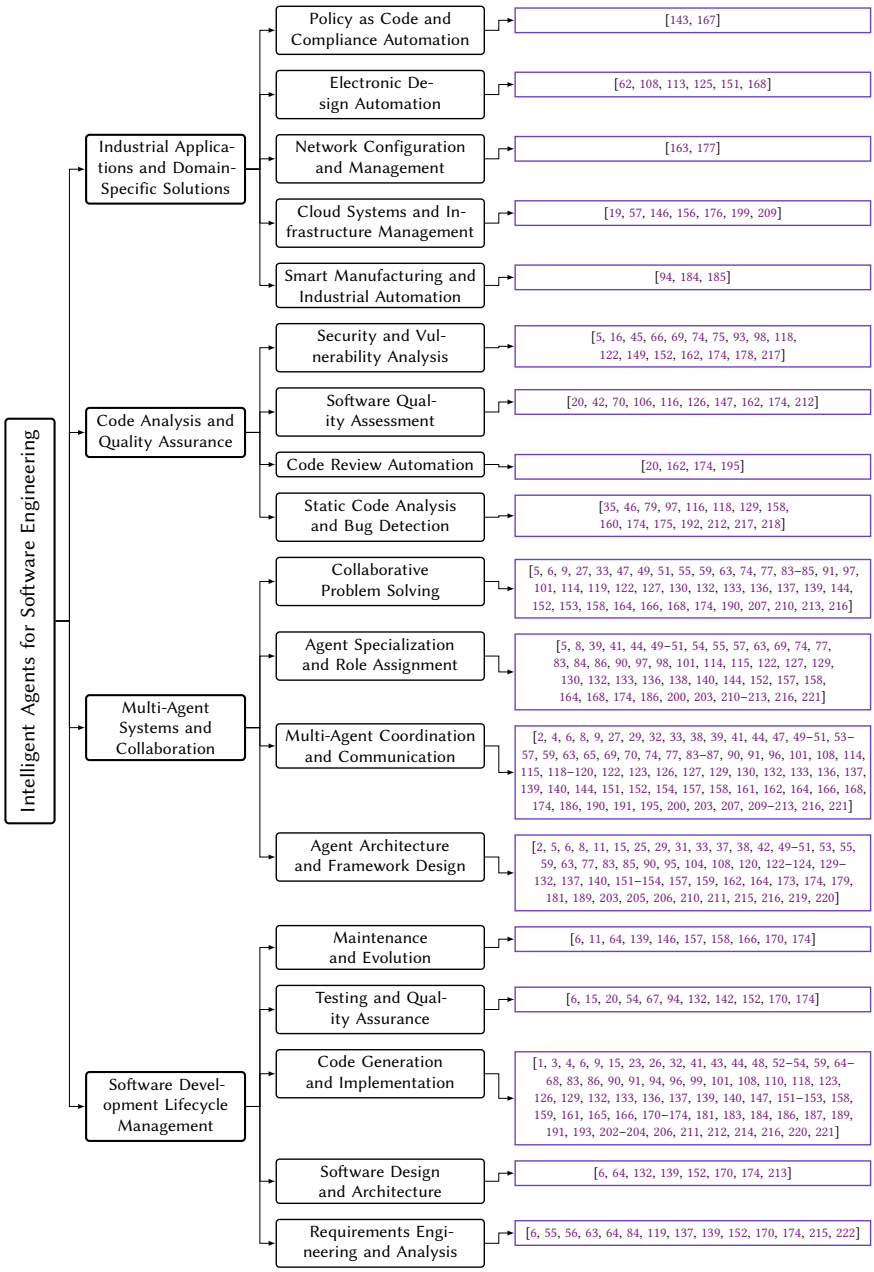


Fig. 4. Classification of Intelligent Agents for Software Engineering (Part I)

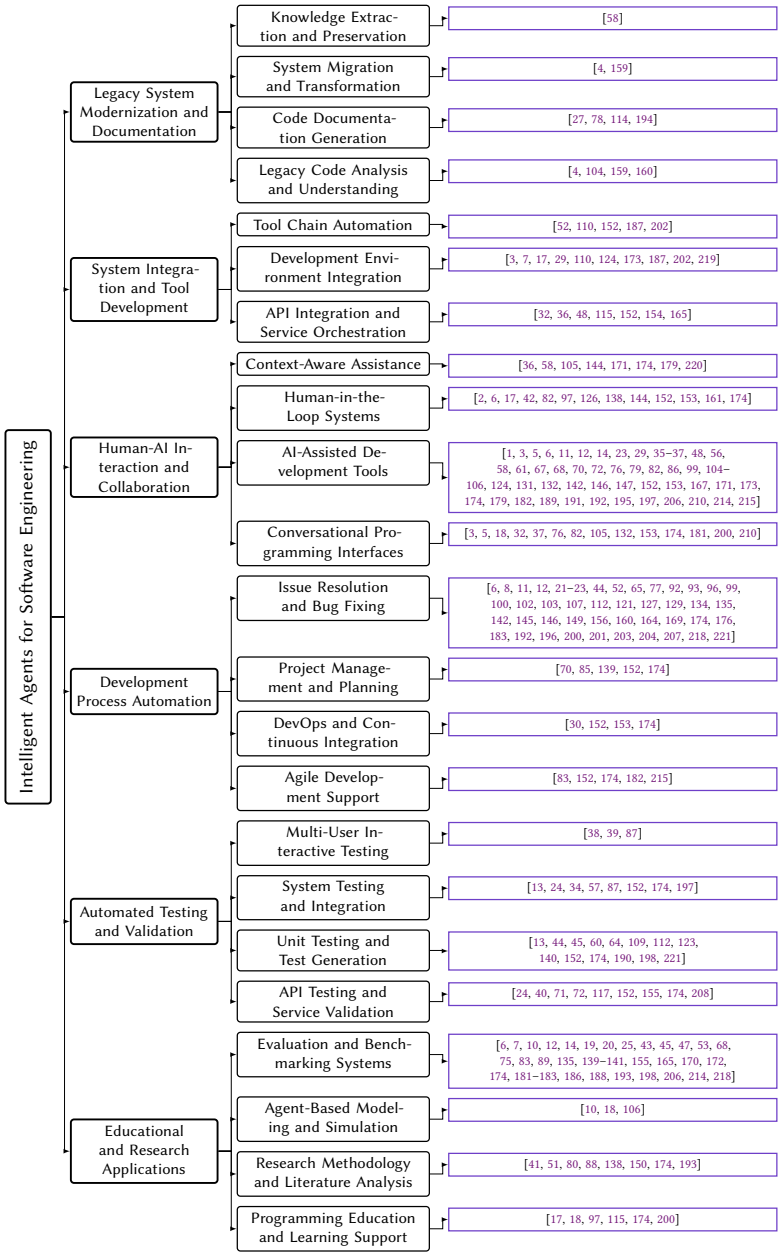
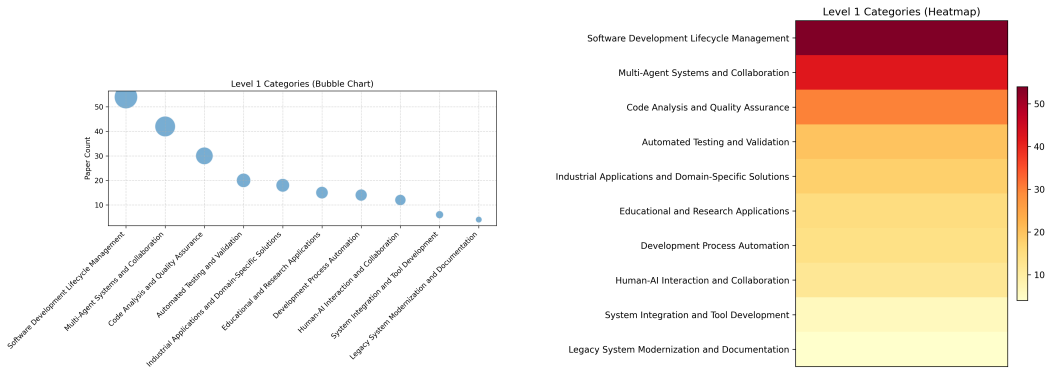


Fig. 5. Classification of Intelligent Agents for Software Engineering (Part II)

2.4.2 Classification Analysis and Distribution. Our quantitative analysis reveals significant research concentration patterns and underexplored areas. Table 1 provides the detailed distribution across Level 1 categories.

Table 1. Distribution of publications across Level 1 categories (N = 215).

Category	**Count**	**Percentage (%)**
Software Development Lifecycle Management	54	25.1
Multi-Agent Systems and Collaboration	42	19.5
Code Analysis and Quality Assurance	30	14.0
Automated Testing and Validation	20	9.3
Industrial Applications and Domain-Specific Solutions	18	8.4
Educational and Research Applications	15	7.0
Development Process Automation	14	6.5
Human-AI Interaction and Collaboration	12	5.6
System Integration and Tool Development	6	2.8
Legacy System Modernization and Documentation	4	1.9
Total	215	100.0



(a) Bubble chart (b) Heatmap

Fig. 6. Publication distribution across Level 1 categories. (a) Bubble chart: bubble size ∝ paper count; (b) Heatmap: color intensity ∝ paper count, emphasizing relative focus.

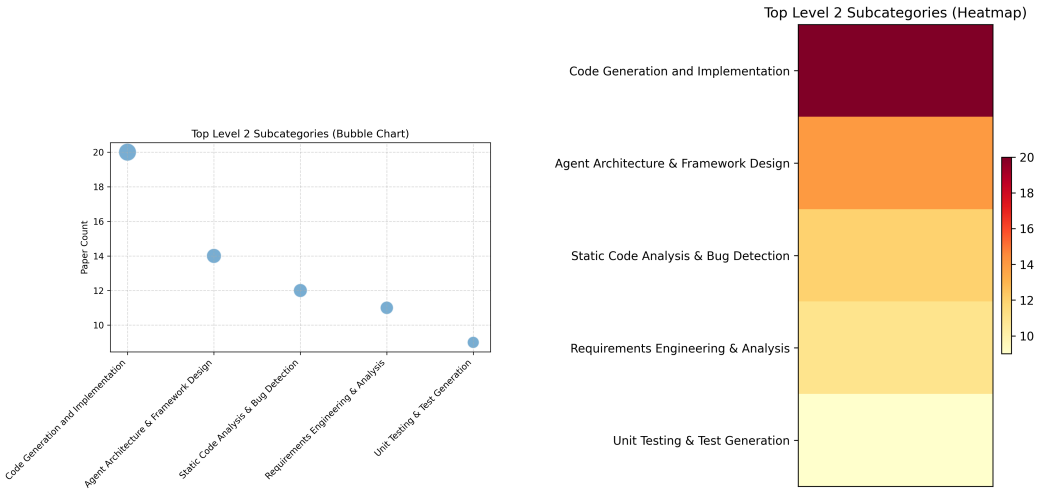
Key Distribution Patterns. Software Development Lifecycle Management dominates with 54 papers (25.1%), reflecting sustained focus on core development phases. *Multi-Agent Systems and Collaboration* follows with 42 papers (19.5%), indicating strong interest in coordinated agent frameworks. Together with *Code Analysis and Quality Assurance* (30 papers, 14.0%) and *Automated Testing and Validation* (20 papers, 9.3%), these four categories account for nearly 70% of all studies.

Conversely, *System Integration and Tool Development* (6 papers, 2.8%) and *Legacy System Modernization and Documentation* (4 papers, 1.9%) represent clear research gaps, totaling only 4.7% of the corpus. This distribution pattern highlights both mature research areas and emerging opportunities for future investigation.

Level 2 Subcategory Analysis. Table 2 shows the most studied Level 2 subcategories, with the top five accounting for 66 papers (30.7% of the corpus).

Table 2. Top five Level 2 subcategories by paper count.

Subcategory	Count	Percentage (%)
Code Generation and Implementation	20	9.3
Agent Architecture & Framework Design	14	6.5
Static Code Analysis and Bug Detection	12	5.6
Requirements Engineering and Analysis	11	5.1
Unit Testing and Test Generation	9	4.2



(a) Bubble chart (b) Heatmap

Fig. 7. Publication distribution across the top five Level 2 subcategories. (a) Bubble chart shows absolute counts; (b) Heatmap highlights relative intensity.

Code Generation and Implementation leads with 20 papers (9.3%), driven by LLM-powered code assistants and agent-based synthesis frameworks. *Agent Architecture & Framework Design* (6.5%) and *Static Code Analysis and Bug Detection* (5.6%) follow, illustrating concentrated research effort on code synthesis, agent framework design, and automated quality assurance—the foundational pillars for intelligent software engineering systems.

Within SDLC Management, *Maintenance and Evolution* remains underexplored with only 8 papers (3.7%), suggesting opportunities for agent-driven refactoring and evolution planning. Similarly, in Multi-Agent Systems, infrastructure work dominates over specialized applications like collaborative problem-solving.

This classification framework reveals that nearly half of all papers (48.4%) address core code activities, while integration and legacy topics receive limited attention (4.7%), pointing to clear directions for future research as discussed in Section 5.

2.5 Research Questions

The systematic formulation of precise, focused research questions provides the analytical framework that guides data extraction, synthesis procedures, and evidence interpretation [128]. We identified

the critical need to systematically examine the role, implementation, and evolution of Intelligent Agents within software engineering contexts, addressing significant knowledge gaps revealed through preliminary literature analysis.

Our research framework, conceptualized in Figure 8, organizes the investigation around three complementary analytical dimensions that systematically address the current state, technological foundations, and evolutionary trajectory of Intelligent Agents in software engineering.

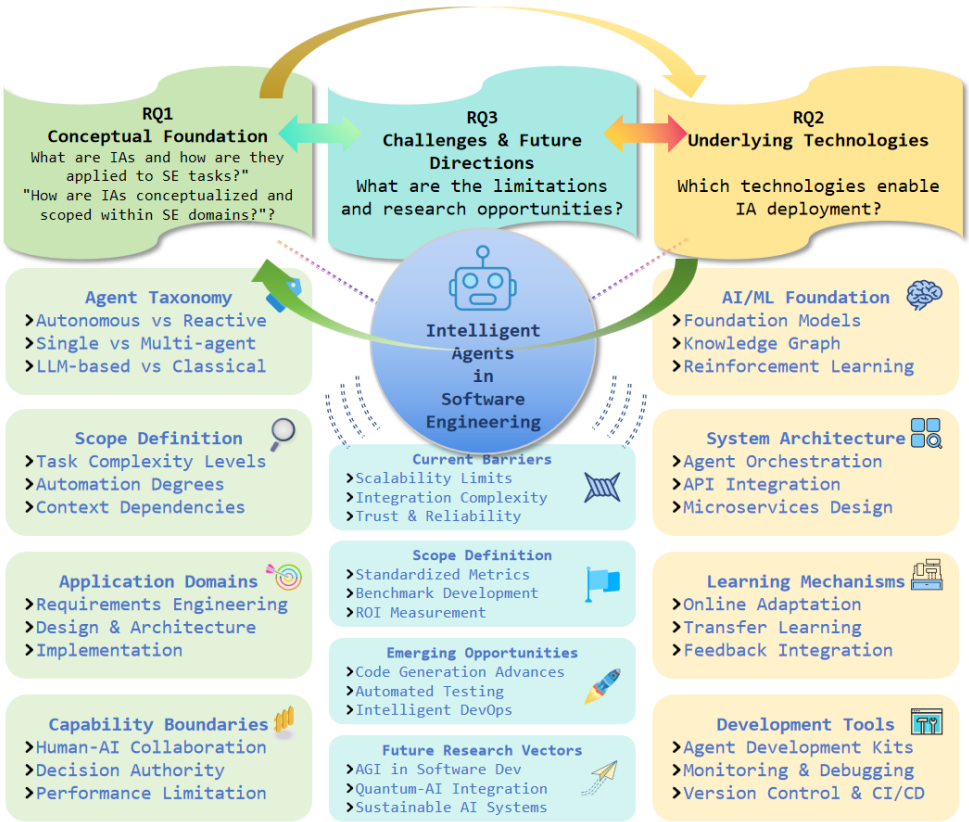


Fig. 8. Research Framework Conceptual Map. The three-dimensional framework organizes the systematic investigation around: RQ1 (Conceptual Foundation & Scope), RQ2 (Underlying Technologies), and RQ3 (Challenges & Future Directions), with bidirectional relationships indicating methodological interdependencies.

The research questions were systematically designed to address three critical analytical dimensions that collectively provide comprehensive understanding of Intelligent Agents in software engineering. Each question was methodically crafted to ensure both independent analytical value and synergistic contribution to overall understanding.

The systematic investigation addresses the following methodologically structured research questions:

(1) **RQ1: How are Intelligent Agents conceptualized and what is the scope of their current applications in software engineering contexts?**
This foundational research question systematically addresses the conceptual landscape of Intelligent Agents within software engineering domains, examining both theoretical frameworks

and practical implementation patterns. The investigation encompasses systematic analysis of definitional approaches, taxonomic classifications, and architectural paradigms employed across the literature, including agent taxonomy development, application domain mapping across software engineering lifecycle phases, and capability boundary analysis examining human-AI collaboration patterns.

(2) **RQ2: What technological architectures and implementation approaches enable Intelligent Agent deployment in software engineering applications?**
This technically-focused research question systematically examines the technological substrates, architectural patterns, and implementation methodologies that enable Intelligent Agent capabilities in software engineering contexts. The investigation encompasses comprehensive analysis of AI/ML foundations, system architectural approaches, learning mechanisms, and development tool ecosystems, including foundation technologies, system architecture patterns, learning mechanism analysis, and development tool examination.

(3) **RQ3: What are the primary barriers to Intelligent Agent adoption in software engineering, and what opportunities exist for future development and research?**
This forward-looking research question systematically analyzes current limitations, deployment challenges, and evolutionary opportunities that will shape the future development of Intelligent Agents in software engineering. The investigation encompasses systematic analysis of current barriers, emerging opportunities, evaluation challenges, and future research directions, providing strategic insights for both researchers and practitioners.

These systematically formulated research questions collectively provide a comprehensive analytical framework for examining Intelligent Agents in software engineering contexts. The methodological structure ensures systematic coverage of conceptual foundations, technological implementations, and evolutionary trajectories while maintaining analytical coherence and enabling evidence-based conclusions.

3 RQ1: WHAT ARE INTELLIGENT AGENTS, AND WHICH OF THESE AGENTS ARE CURRENTLY APPLIED TO SPECIFIC TASKS IN SOFTWARE ENGINEERING?

3.1 Conceptualizing Intelligent Agents

The proliferation of Intelligent Agents in software engineering has been accompanied by considerable conceptual ambiguity, with the literature sometimes conflating sophisticated autonomous systems with simpler chatbots, rule-based linters, or procedural scripts. Establishing precise definitional boundaries is therefore fundamental for rigorous systematic analysis.

Canonical Definition and Theoretical Foundation. Drawing upon foundational artificial intelligence research [148, 180] and its contemporary reinterpretation for large language model-centric development workflows [80, 139], we establish a canonical definition:

We define an **Intelligent Agent for software engineering** as:

> *A software entity that (i) systematically **perceives** software engineering artefacts and development events through dedicated sensory mechanisms, (ii) maintains a persistent internal **state** comprising beliefs, objectives, and accumulated knowledge, (iii) employs sophisticated **reasoning** processes to formulate multi-step plans that optimise task-specific utility functions, and (iv) **acts** upon the development environment through programmatic effectors—while operating with significant **autonomy** under dynamic feedback conditions.*

This definition establishes five indispensable properties, as illustrated in Figure 9:

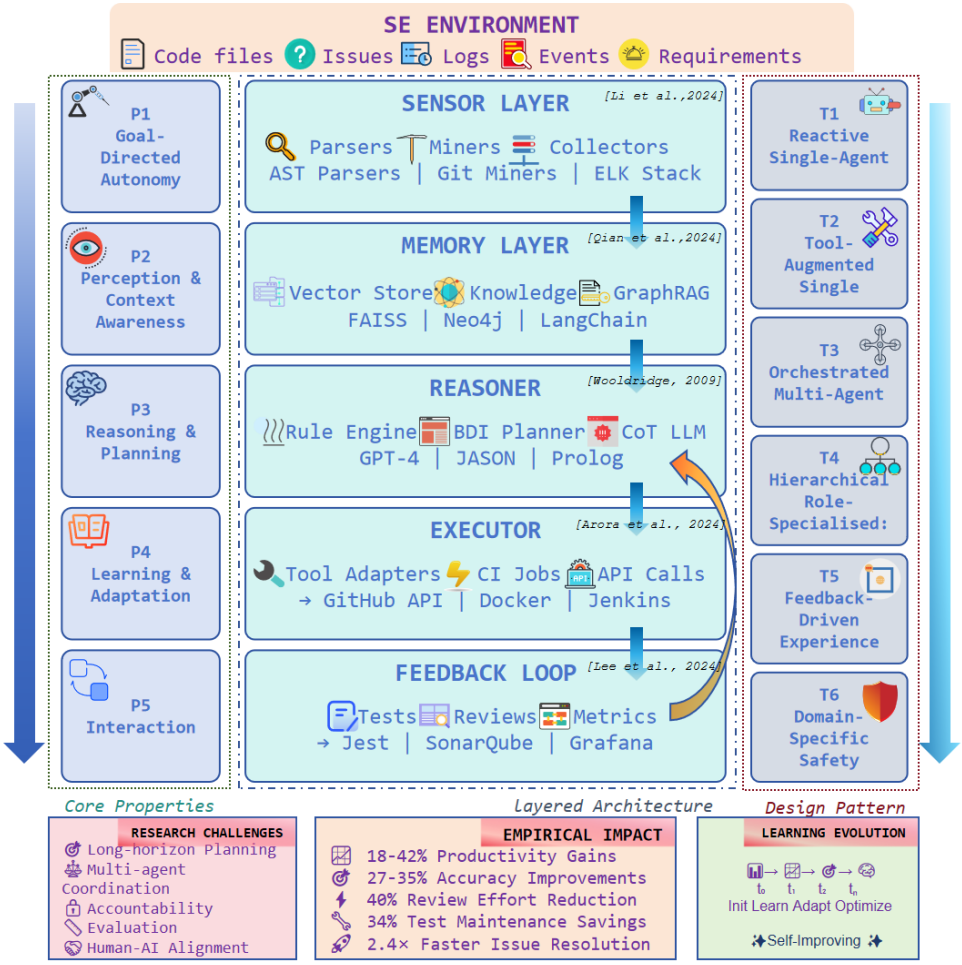


Fig. 9. Intelligent Agent Architecture in Software Engineering Environment. The framework demonstrates the five core properties (P1-P5), layered architectural components, six design patterns (T1-T6), and empirical evidence of agent impact.

P1 Goal-Directed Autonomy: Agents pursue explicitly defined objectives and possess the capability to independently determine when, whether, and how to invoke available tools [49].

P2 Perception and Context Awareness: Agents continuously monitor heterogeneous software engineering artefacts while constructing comprehensive situational awareness [132].

P3 Reasoning and Planning: Agents employ sophisticated deliberation mechanisms to decompose high-level objectives into actionable steps [185].

P4 Learning and Adaptation: Agent performance demonstrates measurable improvement through systematic incorporation of feedback signals [131].

P5 Social Interaction and Communication: Agents engage in meaningful communication with human developers and peer agents through natural language interfaces [51].

Architectural Framework. The prevailing design pattern organizes agent functionality into five interconnected architectural layers, each serving distinct roles in enabling intelligent behavior:

- **Sensor Layer**: Specialized components for parsing source code, mining repository data, and collecting operational telemetry.
- **Memory Layer**: Persistent knowledge storage through vector databases and knowledge graphs supporting RAG patterns [80].
- **Reasoning Engine**: Diverse deliberation mechanisms including rule-based engines, BDI planners, and chain-of-thought prompting [176].
- **Execution Layer**: Tool adapters enabling concrete actions including code editing, compilation, and CI/CD integration [37].
- **Feedback Loop**: Systematic mechanisms for capturing performance signals and enabling continuous learning [81].

Distinguishing Characteristics. The fundamental distinction between Intelligent Agents and traditional automation tools lies in their operational characteristics. Conventional tools operate in a *stateless* manner, serve *single-purpose* functions, and remain *non-adaptive*. In contrast, Intelligent Agents demonstrate superior performance in contexts requiring adaptive responses, long-horizon reasoning, and iterative self-improvement.

Empirical validation provides compelling evidence: agent-based pipelines reduce human code review effort by up to 40% [139] and achieve 34% reductions in test maintenance costs [37]. These improvements reflect the fundamental advantages of adaptive, context-aware automation.

3.2 Agent Typologies Applied in Software Engineering

Our systematic analysis of 215 primary studies reveals six distinct agent typologies (T1-T6) that vary in autonomy levels, functional specialization, and organizational complexity. These patterns, illustrated in Figure 9, provide a structured lens for understanding agent deployment across different application domains.

T1 — Reactive Single-Agent Assistants. Reactive single-agent systems operate as sophisticated copilots within established development environments. These agents embody a monolithic architecture where LLMs serve as centralized reasoning engines responding to developer prompts. Representative implementations include autonomous testing assistants [37] and industrial control agents [185]. While offering minimal deployment complexity and immediate accessibility, their reactive nature limits capacity for long-horizon planning and autonomous task decomposition.

T2 — Tool-Augmented Single Agents. Tool-augmented agents extend the conversational paradigm with systematic integration of external development tools. This architectural enhancement enables direct environmental manipulation and autonomous verification through closed-loop feedback mechanisms. Rasheed et al. demonstrate comprehensive end-to-end development pipelines [139], while Wang et al. apply this paradigm to cloud infrastructure management [176]. The tool-augmentation approach reflects broader trends toward grounded, embodied agents [80].

T3 — Orchestrated Multi-Agent Assemblies. Orchestrated multi-agent assemblies distribute specialized capabilities across multiple autonomous units coordinating through structured communication protocols. ChatDev exemplifies role-oriented specialization with Designer, Developer, and Tester agents [132], while MAGIS incorporates Repository Custodian and QA Engineer roles for GitHub workflows [164]. Empirical evaluations demonstrate superior performance through division of labor, with measurable improvements in scalability and reduced hallucination rates [51].

T4 — Hierarchical Multi-Agent Systems. Advanced architectures incorporate explicit hierarchical organization enabling sophisticated task decomposition and resource allocation. MASAI implements distinct sub-agents for information retrieval, planning, patch generation, and testing [8], achieving state-of-the-art performance on SWE-bench Lite. Händler's taxonomy provides theoretical foundations for balancing autonomy and alignment across hierarchical layers [49]. These systems excel in scenarios requiring complex quality assurance and systematic output integration.

T5 — Feedback-Driven Experience-Refining Agents. T5 agents implement explicit feedback loops enabling systematic learning from operational experience. The Iterative Experience Refinement paradigm demonstrates how agents harvest execution trajectories and apply learned patterns [131]. FixAgent illustrates practical application through diagnosis-patch-validation cycles, fixing 2.5 times more issues than single-pass approaches [77]. These systems integrate multiple feedback sources including automated testing, static analysis, and human reviews for comprehensive learning.

T6 — Domain-Specific Safety-Critical Agents. The most sophisticated implementations are tailored to specialized domains with stringent requirements for reliability and compliance. Static analysis agents reduce false positive rates through domain-specific pattern recognition [35]. Smart contract auditing agents achieve 91% F1 scores while providing interpretable explanations [98]. Dependency reasoning agents address supply chain security through knowledge graph mining [5]. Responsible AI architectures emphasize comprehensive traceability and audit mechanisms [95].

Taxonomic Integration. The six typologies represent a progression of increasing sophistication reflecting diverse software engineering requirements. Reactive agents (T1) provide accessible entry points with immediate productivity benefits. Tool-augmented agents (T2) enable autonomous verification capabilities. Orchestrated (T3) and hierarchical (T4) systems leverage complementary competencies for complex projects. Feedback-driven agents (T5) enable continuous improvement over extended deployments. Domain-specific agents (T6) demonstrate adaptability to specialized contexts with stringent requirements.

This taxonomic framework provides strategic guidance for selecting appropriate architectures, representing a maturation pathway from reactive assistance through collaborative orchestration to specialized domain expertise. The progression aligns with organizational capability development and increasing automation sophistication, establishing the foundation for comprehensive task-oriented analysis in Section 3.3.

3.3 Mapping Agents to Software Engineering Tasks

Building upon the conceptual foundations and taxonomic framework established in the preceding sections, we now examine how Intelligent Agents are deployed across specific software engineering tasks. This mapping reveals how the six agent typologies (T1-T6) are instantiated in practice, demonstrating their adaptability to diverse development contexts and requirements.

3.3.1 Software Development Lifecycle Management.

End-to-end orchestration. The most ambitious applications of Intelligent Agents target comprehensive SDLC automation, leveraging the full spectrum of capabilities from perception through action. Rasheed et al. present an autonomous GPT-4 pipeline that iteratively refines user stories, generates code, and validates builds, demonstrating the feasibility of integrated agentic development [139]. Building on this vision, Wang's survey frames the SDLC as a sequence of perceiving–planning–acting loops, outlining how tool-augmented LLM agents can self-schedule tasks, trigger CI jobs, and monitor runtime metrics [174]. These holistic approaches promise reduced

context-switching and faster feedback, yet they also surface challenges in long-horizon planning and error recovery.

Phase-specific specialisation. To address the complexity of full-cycle automation, many researchers decompose the SDLC into specialised agent roles. Qian's `ChatDev` separates *Designer* and *Developer* agents to translate natural-language requirements into UML drafts and then into compilable code [132]. Jin et al. extend this approach with an LLM-powered *Maintainer* that performs impact analysis and semantic refactoring [64], while Huang introduces knowledge-guided agents for test-case generation and coverage optimisation [55]. Empirical evaluations report productivity gains ranging from 18% to 42%, albeit coupled with rising coordination overhead, which unified frameworks such as `USEA` aim to mitigate [6].

3.3.2 Multi-Agent Systems and Collaboration.

Architectural blueprints. Multi-agent research in software engineering focuses on designing robust organisational patterns that leverage the collaborative capabilities identified in our taxonomic analysis. Händler proposes a taxonomy that balances autonomy, alignment, and task granularity, serving as a design space for agent orchestration [49]. MAGIS instantiates this blueprint with a manager–worker hierarchy that assigns specialised roles (retriever, planner, coder, tester) and fuses their outputs via a voting mechanism, achieving superior bug-fix accuracy on SWE-bench Lite [164]. These studies highlight the benefits of division of labour, while underscoring the need for conflict-resolution and traceability mechanisms.

Coordination and adaptability. Effective collaboration hinges on rich communication protocols and continual learning—key properties identified in our conceptual framework. He et al. demonstrate that role-aware messaging combined with explicit task contracts reduces hallucination rates in complex pipelines [51]. Arora's `MASAI` further adds self-reflection hooks, enabling worker agents to critique and revise their outputs based on peer feedback [8]. Long-running projects benefit from self-evolving workflows: Sami reports a lightweight nexus architecture that dynamically spawns or retires agents according to backlog pressure [152], while Liu's `SEW` framework leverages execution logs to retrain coordination policies on-the-fly [90]. Recent knowledge-guided pipelines aim to generalise these adaptations across domains [55].

3.3.3 Code Analysis and Quality Assurance.

Broad landscape of analysis agents. Across the 215 primary studies, nearly one quarter target code understanding and assurance, demonstrating the critical role of Intelligent Agents in maintaining software quality. Survey evidence indicates a rapid shift from heuristic linters toward LLM-driven agents able to parse full repositories, reason about execution traces, and suggest remediations [174]. Early static-analysis agents such as ICAA demonstrate sizeable false-positive reductions by coupling pattern matching with LLM reasoning [35], while AutoSafeCoder embeds a multi-agent RAG loop to harden security-critical code paths before release [118]. Together these works highlight a move from point diagnostics to goal-oriented agents that iteratively explore, verify, and patch defects.

Specialised assurance roles. Task-specific agents further refine the quality assurance pipeline through targeted capabilities. CodeAgent introduces a supervisor–worker ensemble that debates patch relevance during review, outperforming single-shot baselines on four core review tasks [162]. Concept-guided agents integrate safety ontologies to score design trade-offs and generate compliance evidence for human auditors [42]. In vulnerability analysis, iAudit combines fine-tuned detectors with debating agents to justify Solidity smart-contract flaws, achieving 91% F1 [98]. VulnBot extends this paradigm to penetration testing, autonomously chaining exploit generation

with mitigation suggestions [74]. Collectively, these agents illustrate how modular reasoning, debate, and tool use deliver measurable gains in precision and developer trust.

3.3.4 *Industrial Applications and Domain-Specific Solutions.*

Automation and infrastructure. Industry-facing studies adapt agent frameworks to cyber-physical settings where latency, safety, and legacy interfaces dominate requirements—exemplifying the T6 domain-specific agents in our taxonomy. In smart manufacturing, GPT-4 agents orchestrate digital-twin production lines, planning task sequences and executing PLC skills on the shop floor [185]. Agents4PLC closes the loop by reading sensor traces and rewriting ladder logic for defect recovery in milliseconds [94]. Cloud operations embrace similar autonomy: RCAgent diagnoses root-cause incidents via tool-augmented log mining [176], while cross-vendor evaluations confirm agent-based RCA rivals seasoned engineers on unseen outages [146]. Reconfigurable MAS controllers then scale these insights to micro-service fleets, dynamically instantiating or retiring agents as SLO pressure fluctuates [57].

Domain-tailored expertise. Networks, hardware, and compliance each foster bespoke agent skills that demonstrate the adaptability of our architectural framework. Net-Translate agents convert natural-language intents into vendor-agnostic configurations, cutting rollout time by 62% in campus test-beds [177]. In electronic-design automation, Coopetitive-V promotes cooperative and adversarial agent teams to optimise RTL pipelines, surpassing state-of-the-art synthesis flows on timing closure [108]. Finally, policy-as-code agents such as PromptPDL parse statutory text, generate formal checks, and embed RAG justification into CI gates, reducing audit preparation from days to hours [167]. These successes underline the adaptability of agentic patterns to stringent, domain-specific constraints while maintaining explainability and human oversight.

3.3.5 *Educational and Research Applications.*

Learning support and tutoring. Agents are increasingly positioned as personalised instructors that leverage their perception and reasoning capabilities to generate examples, hints, and formative feedback in real time. `ChatLogo` extends the classic LOGO environment with an LLM agent that interprets natural-language queries and synthesises turtle-graphics code, improving novice task-completion rates by 27% in classroom studies [18]. `AIera` introduces a multi-agent teacher–student model in which a *Mentor* decomposes programming concepts while a *Coach* adapts difficulty based on learner embeddings, yielding significant learning-gain effect sizes over static MOOCs [97]. Complementing these designs, `vTutor` fuses a retrieval-augmented knowledge graph with an LLM to answer SDG-themed coding questions, whereas `CoLearning` embraces peer-agent debate to surface alternative solutions, collectively demonstrating the versatility of agent-based pedagogy [17, 200].

Research analytics and benchmarking. Beyond pedagogy, agents automate literature curation and experimental reproducibility through systematic perception and analysis capabilities. Sami et al. prototype an agentic SLR pipeline that scrapes metadata, screens abstracts, and extracts statistics with BERT-based filters, cutting manual effort by 60% [150]. `ResearchCodeAgent` leverages role-specialised agents to mine repositories and build citation-aware knowledge graphs, enabling meta-analyses of code-artifact evolution [41]. To foster rigorous evaluation, HumanEvalComm augments unit-test benchmarks with communication metrics for agent ensembles [181], while `LMR-Bench` introduces continual-learning suites that track degradation across agent updates [193]. Collectively, these systems highlight how Intelligent Agents advance both the conduct and the appraisal of SE research.

3.3.6 *Automated Testing and Validation.*

Fine-grained test generation. Recent work equips agents with tool adapters that compile code, execute unit tests, and amplify API suites—demonstrating the practical application of tool-augmented architectures (T2). `Experimenter` shows that a manager–worker ensemble can outperform single-LLM baselines on REST-API coverage by orchestrating request synthesis, oracle inference, and amplification [152]. For service validation, `REST-Agent` coordinates crawler, schema-miner, and verifier agents to generate contract-based tests, detecting 31 previously unknown faults in open-source micro-services [71]. Complementarily, `Test-Amp` demonstrates that replay-guided amplification agents raise mutation-score improvements by up to 22% on industrial APIs [117].

System-level and interactive testing. At broader scopes, autonomous LLM agents synthesise end-to-end scenarios and emulate concurrent users, leveraging their reasoning and planning capabilities. Yoon et al. show that a planning-and-execution agent can configure distributed environments, invoke system calls, and triage logs to validate cloud-native stacks [197]. `SLA-MAS` extends this concept with a reconfigurable MAS that spawns specialised load oracles when service-level objectives drift [57]. For UX assurance, `Agent-UserTest` pairs an *Action Planner* with a *Feedback Collector* to simulate multi-user sessions, revealing latent race conditions in collaborative editors [38]. Breaking the single-tester limit, a consensus-driven agent swarm uncovers 45% more GUI anomalies than scripted Selenium baselines [39].

3.3.7 Development Process Automation.

Agile planning and DevOps. Agents streamline iterative development by negotiating sprint goals, updating backlogs, and triggering CI/CD workflows—embodying the goal-directed autonomy central to our definition. `Enhance-Agent` attaches a *Scrum Master* role to an LLM that reprioritises stories and recomputes velocity forecasts, reducing planning-meeting duration by one-third [215]. `AgentOps` integrates observability pipelines with GPT-4 to diagnose build failures and roll back faulty commits within minutes, achieving 93% precision in root-cause localisation [30]. Complementary field studies report that an evaluation-driven agentic loop accelerates DevOps feedback by 26% on large repositories [182].

Issue triaging and self-repair. Downstream of CI, specialised agents tackle bug localisation and automated program repair through iterative refinement—exemplifying T5 feedback-driven agents. `RCAgent` chains log analysis, anomaly detection, and patch suggestion to close incident tickets 2.4× faster than manual workflows [176]. MAGIS extends this paradigm with a manager–worker hierarchy that fixes SWE-bench issues by combining retrieval, patching, and regression testing agents [164]. On low-resource languages, `LANTERN` translates Rust defects to Python, applies multi-agent refinement, and ports the patch back, fixing 37% more bugs than monolingual baselines [96]. Finally, graph-guided agents such as `LocAgent` exploit dependency graphs to recommend context-aware edits, surpassing heuristic refactorers on ten open-source projects [22]. These results illustrate how agentic collaboration augments traditional automation, pushing process efficiency and repair accuracy to new frontiers.

3.3.8 Human–AI Interaction and Collaboration.

Conversational interfaces and assistive tooling. A surge of studies equips LLM agents with natural-language front ends that let developers articulate intentions in plain English and receive code, tests, or visual artefacts in response—leveraging the social interaction capabilities identified in our framework. `ChatDev` demonstrates a multi-role dialogue that converts requirements into compilable projects through iterative discussion [132]. `ChatLogo` adapts the idea to introductory programming, allowing students to verbalise turtle-graphics goals and instantly preview generated scripts [18]. Industrial surveys confirm that conversational copilots embedded in IDEs or GIS tools materially

shorten context-switching time and raise perceived developer satisfaction [37, 76, 174]. Beyond text, agents like Text2BIM map free-form descriptions to 3-D building models, hinting at multimodal collaboration horizons [32].

Human-in-the-loop refinement and context awareness. The most effective agents support incremental, bidirectional refinement rather than one-shot generation, embodying the learning and adaptation properties of our conceptual model. AIera frames teaching as a mentor–student dialogue where the mentor agent adapts explanations after learner feedback, achieving superior knowledge retention [97]. Field experiments in enterprise settings show that embedding acknowledgement steps—e.g. the *review* function in HumanEvalComm—reduces erroneous code insertions by 35% [181]. Hybrid workflows further integrate runtime telemetry so that context-aware agents proactively surface relevant files or spreadsheet formulas as users navigate artefacts [105]. Such adaptive loops strengthen trust and mitigate over-automation risks documented in large-scale collaborations [144, 161].

3.3.9 System Integration and Tool Development.

API composition and orchestration. Several projects treat the agent as a glue layer that discovers, links, and supervises heterogeneous services—demonstrating advanced perception and execution capabilities. CodeNav illustrates this pattern by letting an LLM agent parse OpenAPI specs, generate integration snippets, and validate call graphs within real-world micro-service repositories [48]. Complementarily, AppWorld exposes a controllable sandbox where agents wire mobile app components through natural-language commands, accelerating prototype assembly [165]. Design guidelines emerging from cross-case syntheses emphasise standardised message schemas and discovery protocols as enablers of scalable orchestration [36, 154].

IDE embedding and tool-chain automation. Deep integration into development environments pushes agent actions from "advice" to "direct manipulation," fully realising the action capabilities in our architectural framework. AI2Apps grafts a visual agent onto a no-code IDE, allowing drag-and-drop widgets to be parameterised via LLM reflections [124]. AutoGen Studio offers a sandbox where agents can instantiate new tools on demand, fostering recursive workflow construction [29]. Empirical bench-marking shows that agents capable of installing and configuring third-party libraries autonomously (e.g. pip-Install agent) resolve environment issues 48% faster than scripted CI fixes [110]. When extended with repository-level compilation skills, agents like CompileAgent achieve end-to-end build success on 79% of 200 large-scale projects without human patches [52], highlighting the maturation of agentic tool-chain automation.

3.3.10 Legacy System Modernization and Documentation.

Analysis and migration of ageing code bases. Modernising legacy stacks demands agents that can parse obsolete languages, infer architecture, and suggest migration paths—requiring the full spectrum of perception, reasoning, and action capabilities. LegacyWebAgent applies static-dynamic hybrids to PHP/ASP monoliths, automatically partitioning modules before recommending micro-service targets [4]. In automotive software, LLM-powered agents query megabyte-scale codebases to map feature interactions, reducing onboarding time for new developers by half [104]. Cross-language translation agents further enable Cobol-to-Java or ROS1-to-ROS2 migrations with semantic preservation guarantees [159].

Documentation, knowledge capture, and preservation. Sustaining legacy systems also hinges on high-fidelity documentation, where agents apply their communication and reasoning capabilities. TeamWorkMakesDreamWork orchestrates a reviewer-writer agent pair to transform sparse

comments into task-oriented design documents, improving comprehension scores in controlled studies [114]. Debate-driven agents generate multi-view explanations—algorithmic, architectural, historical—yielding richer documents than single-speaker summaries [27]. Domain-tuned generators such as `DocAgent` outperform baseline summarisation on Cobol programs by leveraging copy-book structure cues [194]. Finally, onboarding assistants extract tacit tribal knowledge into searchable graphs, ensuring institutional memory survives turnover [58].

RQ1 – Summary

(1) **Conceptual definition.** An Intelligent Agent for software engineering is a software entity that *perceives* SE artefacts and events, maintains an internal belief–goal state, *reasons* to devise utility-maximising plans, and *acts* autonomously on the development environment—closing the loop through dynamic feedback.
(2) **Essential properties.** Five indispensable traits—goal-directed autonomy, context awareness, reasoning & planning, continual learning, and social interaction—distinguish agents from stateless bots or scripts.
(3) **Agent typologies.** Six recurring designs were identified: (T1) reactive single agents, (T2) tool-augmented single agents, (T3) orchestrated multi-agent ensembles, (T4) hierarchical/role-specialised MAS, (T5) feedback-driven experience-refining agents, and (T6) domain-specific or safety-critical agents.
(4) **Task-oriented deployment.** Agents now permeate ten SE task families, from end-to-end SDLC orchestration and collaborative MAS frameworks to domain-tailored applications in cloud operations, manufacturing, education, and legacy modernisation. Tool use, role specialisation, and natural-language coordination re-appear as cross-cutting enablers.
(5) **Empirical impact.** Reported benefits include 18–42 % productivity gains, 27–35 % accuracy improvements, and substantial reductions in false positives and incident resolution time. Challenges persist in long-horizon planning, multi-agent coordination overhead, and rigorous, standardised evaluation.
(6) **Research gaps.** Open issues involve benchmarking complex interactions, aligning autonomous decisions with human intent, extending support to legacy ecosystems, and establishing safety and accountability guarantees for agentic pipelines.

4 RQ2: WHICH INTELLIGENT AGENTS-BASED TECHNOLOGIES ARE USED IN EACH TASK CATEGORY?

Having established the conceptual foundations, taxonomic classifications, and task-oriented deployments of Intelligent Agents in software engineering through RQ1, we now turn to the fundamental question of technological enablement: what specific technologies, architectural patterns, and implementation frameworks empower these agents to achieve their documented capabilities across diverse software engineering contexts? This research question addresses the critical gap between conceptual understanding and practical implementation by systematically analyzing the technological substrates that transform theoretical agent capabilities into operational software engineering solutions.

The technological landscape of Intelligent Agents in software engineering exhibits remarkable convergence around a core set of universal building blocks, despite the apparent diversity of application domains and use cases. As illustrated in Figure 10, our comprehensive analysis of 215 primary studies reveals that Intelligent Agent implementations consistently employ four

fundamental technological components that form the architectural foundation for successful deployments across all task categories.

Fig. 10. Technological Framework for Intelligent Agents in Software Engineering. The diagram illustrates the universal building blocks at the center (Orchestration Engines with RAG Grounding, CoT/ToT Planning, Tool Integration, and Multi-Agent Coordination) that enable applications across eight primary task categories, along with documented empirical benefits ranging from 5-28pp accuracy improvements to 18-42% productivity gains and comprehensive quality assurance automation.

The technological convergence depicted in Figure 10 demonstrates that successful Intelligent Agents implementations, regardless of their specific application domain, consistently leverage four interconnected technological pillars. Orchestration Engines provide the coordination mechanisms that enable complex multi-step workflows and agent interaction patterns. Retrieval-Augmented Generation (RAG) Grounding ensures that agent reasoning processes are anchored in relevant, contextual information rather than relying solely on parametric knowledge. Chain-of-Thought and

Tree-of-Thought Planning mechanisms enable transparent, traceable reasoning processes that can be audited and debugged. Tool Integration capabilities allow agents to interact meaningfully with existing software engineering infrastructure and development toolchains.

These universal building blocks manifest across eight distinct application categories, as shown in the peripheral sections of Figure 10, each representing a major domain of software engineering practice where Intelligent Agents have demonstrated measurable impact. The documented benefits summarized in the figure reflect systematic empirical evidence from our literature corpus, indicating accuracy improvements ranging from 5-28 percentage points, productivity gains of 18-42%, and significant enhancements in quality assurance automation capabilities.

This technological analysis reveals that the apparent diversity of agent applications masks an underlying architectural unity that provides both theoretical insight and practical guidance for system designers. The recurrence of these technological patterns across disparate application domains suggests that they represent fundamental requirements for Intelligent Agent effectiveness in software engineering contexts, rather than domain-specific optimizations.

Our systematic examination proceeds by analyzing each task category identified in RQ1, examining how these universal building blocks are instantiated, specialized, and optimized for specific software engineering challenges. This approach enables both horizontal understanding of cross-cutting technological themes and vertical understanding of domain-specific implementation patterns and performance characteristics.

The technological survey that follows demonstrates that while Intelligent Agents exhibit remarkable adaptability to diverse software engineering contexts, their effectiveness consistently depends on sophisticated orchestration mechanisms, robust knowledge grounding, transparent reasoning processes, and seamless tool integration. These findings provide actionable guidance for practitioners seeking to implement agent-based solutions while contributing to theoretical understanding of the technological requirements for intelligent autonomy in complex software engineering environments.

4.1 Software Development Lifecycle Management

Scope and significance. Among all Level 1 categories, SDLC management attracts the largest share of studies, reflecting sustained interest in end-to-end, process-aware automation that spans from requirements to maintenance [64, 170, 174].

Common technological substrate. Effective SDLC agents converge on four recurring building blocks: (*i*) orchestration engines for coordinating multi-phase workflows; (*ii*) retrieval-augmented grounding across heterogeneous artifacts; (*iii*) transparent planning (e.g., CoT/ToT) for auditable decisions; and (*iv*) deep tool integration with compilers, test frameworks, and CI/CD [64, 174].

End-to-end orchestration patterns. Vision and platform papers outline unified, role-specialized pipelines that translate stakeholder goals into consistent design/code/test artifacts while preserving traceability: Rasheed et al.'s vision articulates multi-agent SDLC blueprints spanning planning, RE, design and coding [139]; *CodePori* demonstrates staged orchestration from requirements through testing for large projects [137]; *MAO* targets automatic process-model generation via iterative multi-agent refinement and review [84]; *USEagent* unifies repository-level tasks (coding, testing, patching) under a single agentic umbrella [6].

Requirements and upstream phases. Addressing early-phase undercoverage, *MARE* decomposes RE into elicitation, modelling, verification, and specification with shared workspaces and measurable quality gains [63]; *Goal2Story* maps high-level business goals to user stories under privately hosted sLLMs to preserve confidentiality [222]; industrial deployment reports also show that coupling

requirements tools with code assistants can lift suggestion accuracy and task success in practice [56]. Benchmark-level evidence further indicates a coverage imbalance: coding dominates ∼60% of evaluations, while requirements and design receive substantially less attention [170].

Downstream integration and continuous verification. Unified platforms increasingly interleave generation with verification, security and quality checks, and artifact-level feedback loops. Examples include pipelines that pair generation with structured tests and review to detect and correct reasoning errors, turning build-and-test into a continuous optimization cycle [6, 137, 170].

What works—and where gaps remain.

- **Patterns.** Unified orchestration with shared artifact pools improves consistency across phases and reduces context switching; RAG over code, tickets and design assets stabilizes long-horizon decisions; schema-guided handoffs between agent roles raise traceability; continuous verification (tests, reviews) reduces last-mile integration risks [6, 63, 84, 137, 139].
- **Gaps.** (*G1*) *Phase imbalance*: benchmarks and case studies still concentrate on coding; RE and design remain under-evaluated [170]. (*G2*) *Robustness at scale*: long-horizon planning and error recovery across repositories require stronger, adaptive orchestration and failure-aware loops [6, 174]. (*G3*) *Human–AI collaboration*: systematic protocols for negotiation, debate, and role handoff are promising for upstream quality (e.g., user-story quality, RE classification) but under-explored in large programs [119, 215].

Takeaways for practitioners and researchers. Design for orchestration first: plan explicit role boundaries, artifact contracts, and evaluation hooks before tool selection. *Ground widely, then plan transparently*: pair broad RAG with auditable planning to curb drift and ease debugging. *Balance the pipeline*: invest in RE/design evaluations and cross-phase metrics to avoid local optima focused on coding [170].

4.2 Multi-Agent Systems and Collaboration

Scope and motivation. Leveraging multiple specialized LLM-based agents has emerged as a dominant pattern for complex SE tasks that demand long-horizon reasoning, division of labor, and continuous verification. Recent surveys and architectural taxonomies converge on the view that multi-agent collaboration can raise robustness and scalability when paired with explicit coordination and tool grounding [49, 51, 95].

Collaboration paradigms. We observe four recurrent paradigms: (*i*) **role-specialized pipelines**, in which agents emulate SE roles (planner/architect/developer/tester) and communicate via structured dialogues or shared artifacts (e.g., ChatDev; FlowGen) [83, 132]; (*ii*) **modular orchestration**, which instantiates sub-agents with clear objectives and strategies (e.g., MASAI) [8]; (*iii*) **self-organized swarms**, where the number and responsibilities of agents scale with task complexity [59]; and (*iv*) **cross-team orchestration**, which runs multiple teams in parallel to explore alternative solution paths and exchange insights [33]. Complementary platforms and tooling (OpenHands, AI2Apps, AutoGen Studio) lower the friction of building and debugging these workflows [29, 124, 173].

Coordination, communication, and memory. Effective collaboration relies on: (*a*) **communication competence** (asking clarifying questions, resolving ambiguity) [181]; (*b*) **supervision/critique loops**, such as QA-checkers, critic agents, or cognitive-layer controllers that gate progression [5, 77, 162]; and (*c*) **experience management**, where agents capture, reuse, and refine trajectories to avoid repeated mistakes (e.g., Experiential Co-Learning; Iterative Experience Refinement) [130, 131].

Reference architectures highlight planning, memory, action, and security as first-class modules to keep autonomy aligned with accountability [95].

Representative applications. Multi-agent collaboration shows tangible benefits across diverse SE activities: *Repository-level code generation* improves when agents integrate search, symbol navigation, and testing (CodeAgent) [206]; *GitHub issue resolution* benefits from manager–developer–QA role triads (MAGIS) [164]; *code review* achieves state-of-the-art automation via supervisory QA-checkers [162]; *debugging and repair* are strengthened by end-to-end cognitive pipelines and tool-invocation FSMs (FixAgent; RepairAgent) [11, 77]; and *requirements engineering* can be decomposed into elicitation, modeling, verification, and specification with shared workspaces (MARE) [63]. Empirical reports often show sizeable gains over single-agent baselines on SWE-bench/HumanEval-style tasks and repo-level benchmarks [8, 164, 206].

What works—concise patterns.

- **Explicit role schemas** (planner/implementer/tester/reviewer) reduce prompt drift and ease handoffs; process emulation (Scrum/Waterfall/TDD) can stabilize collaboration [83, 132].
- **Hierarchical or supervisory control** (lead/critic/QA) curbs cascading hallucinations and keeps teams aligned to the initial objective [5, 77, 162].
- **Tool-centric grounding** (search, build, test, execution) converts dialogue into verifiable actions at repo scope [173, 206].
- **Experience curation** (trajectory distillation, iterative refinement) improves sample efficiency and robustness over time [130, 131].
- **Design for scalability** via modular sub-agents or self-organization when problem size grows [8, 59].

Limitations and open challenges. (*L1*) **Orchestration overhead & cost.** Long trajectories with many agents inflate latency and token budgets; modular strategies (e.g., MASAI) mitigate but do not eliminate this [8]. (*L2*) **Error propagation.** Pipeline designs can amplify early mistakes; supervisory/critic loops help but remain brittle on multi-step refactoring/repair [11, 77]. (*L3*) **Generalizable communication.** Agents still struggle to ask the "right" questions systematically across tasks, limiting transfer [181]. (*L4*) **Security & accountability.** Few systems thread threat modeling, auditable memory, and access control throughout the stack [95]. (*L5*) **Benchmark fidelity.** Gains on function-level tasks do not always translate to repository-scale maintenance and evolution; broader, standardized evaluations remain needed [51, 206].

Actionable guidance. Start with architecture: fix roles, interfaces, memory, and critique policies before prompt tuning [49, 95]. *Ground aggressively*: couple agents with build/test/running environments and repository-aware tools [173, 206]. *Instrument communication*: measure clarifying-question rate and resolution efficacy to drive improvements [181]. *Invest in experience*: curate and refine trajectory memories to reduce retries and regressions over time [130, 131].

4.3 Code Analysis and Quality Assurance

Scope and significance. This stream encompasses static and dynamic code analysis, fault localization and repair, code review automation, repository-level auditing, and security assessment. Recent systems leverage LLM-augmented analysis to reason across heterogeneous artifacts (source, tests, issues, specs), improving coverage beyond traditional tools while keeping actions verifiable [20, 46, 174].

Static analysis and bug finding. LLM-centered "intelligent code analysis agents" revisit static analysis by combining symbolic signals with natural-language reasoning to surface logic defects and reduce false positives [35]. Agents also classify performance-sensitive configurations by chaining prompts with retrieval, outperforming prior methods on multi-project evaluations [175]. At repository scale, autonomous auditors traverse code paths, validate data-flow hypotheses, and filter hallucinations to raise true-bug yield under tight budget constraints [46]. Execution-free error detection couples LLMs with analysis tools to flag likely runtime failures before running code [79]. Beyond general-purpose code, domain-specific agents align implementations to external specifications (e.g., RFCs) to uncover functional bugs [217].

Code review and quality assessment. Multi-agent reviewers emulate collaborative practices with manager/critic/QA roles, detecting inconsistencies and vulnerability introductions while producing actionable revision guidance [162]. Large-scale audits of agent-generated patches on real GitHub issues show heterogeneous behaviors across agents, highlighting gaps in benchmark coverage and suggesting decomposition of complex issues [20]. Industrial decision workflows (e.g., release readiness) and safety co-design benefit from concept-guided agents that maintain traceability from system models to judgments [42, 70].

Security and vulnerability analysis. Security-oriented frameworks integrate static scanning with fuzzing in iterative loops to reduce vulnerable generations without sacrificing functionality [118]. Agentic reasoning over software dependencies augments advisories with graph-grounded analysis and critic feedback [5]. Emerging benchmarks probe risky execution/generation and computer-use agent safety, revealing susceptibility to unsafe actions and prompt injections and motivating stricter evaluation regimes [16, 45, 75]. Program-repair agents increasingly couple dynamic state inspection with constraint reasoning to localize root causes and craft patches; early results show competitive fix rates on real projects [93]. Sector-specific agents continue to appear (e.g., smart contracts, Android apps), adapting analysis to domain semantics [69, 98, 122].

Education and human-in-the-loop benefits. Teachable-agent settings can synthesize debugging materials and measurably improve novices' debugging outcomes [97]. Practitioner-facing studies emphasize evaluation cost and variability, advocating better HITL protocols and measurement for software-development agents [126].

Evidence from challenging codebases. On systems-scale code, deep research agents that mine semantics and commit history achieve higher crash-resolution rates than standard baselines, underscoring the need for global context acquisition [160]. For kernel-level bugs, repository-scale fault localization remains difficult; targeted enhancements yield consistent but modest gains [218].

What works—concise patterns.

- **Verify while you generate.** Pair generation with static/dynamic checks (tests, fuzzing, validators) to curb error propagation and security regressions [46, 118, 162].
- **Graph- and spec-grounding.** Use dependency KGs and formal/spec artifacts to anchor analysis beyond code text [5, 217].
- **Role structure and critique.** Supervisor/critic QA loops improve review fidelity and enable safe escalation paths [20, 162].
- **Budget-aware orchestration.** Repository-level auditing benefits from memory, hypothesis pruning, and validator gates to balance precision and cost [46].

Limitations and open challenges. (*L1*) **Benchmark fidelity & transfer.** Success on function-level tasks does not guarantee robustness on system- or kernel-scale code; richer, scenario-driven

benchmarks are needed [20, 218]. (*L2*) **Safety at the interface.** Computer-use agents and code agents still execute risky actions or generate unsafe code under subtle prompts; systematic defenses and red-teaming remain incomplete [45, 75]. (*L3*) **Evaluation cost and variance.** End-to-end assessments (build, test, fuzz, review) incur high compute and show evaluator variability, complicating fair comparison [126]. (*L4*) **Accountability and trust.** Integrating analysis tools with agent autonomy is necessary but not sufficient for trustworthy deployment at scale [147].

Actionable guidance. Ground broadly, verify continuously: couple analysis/generation with execution, fuzzing, and validators. *Exploit structure*: encode dependencies, specifications, and histories as first-class context. *Design for audit*: enforce role schemas, logging, and critic checkpoints in review/repair loops. *Test where it hurts*: prioritize repo-scale, safety-critical, and kernel-level evaluations to reveal failure modes early [20, 218].

4.4 Automated Testing and Validation

Scope and significance. This line of work spans unit and API testing, GUI/system testing, repository-level test execution, and safety/robustness evaluation. A consistent pattern is to *close the loop* between generation and validation via multi-agent orchestration, tool grounding (build/run/-coverage), and domain-aware oracles, which together raise realism and fault-finding power over single-pass prompts [13, 45, 87, 174].

Unit testing and oracle construction. End-to-end agentic generators iteratively refine tests using execution feedback and coverage reports (e.g., *TestForge*) [60], while multi-agent pipelines reach consensus to curb hallucinated oracles (e.g., *CANDOR*) [190]. Studies on *generalizability* of planning/debug pipelines across diverse base LLMs (e.g., *AdaCoder*) show that adaptive planning and budget control improve Pass@1 and latency [221]. Self-reflective teams that prune weak outputs (e.g., *CodeCoR*) further stabilize quality under noisy signals [123]. Beyond micro-benchmarks, *UTBoost* augments insufficient real-project tests (SWE-Bench), re-labeling erroneous passes and materially shifting leaderboard standings [198].

API/service testing and intent integrity. Multi-agent REST testing exploits dependency graphs and MARL to explore parameter/value spaces and business logic (e.g., *AutoRestTest*, *LogiAgent*) [71, 208]. Stress-testing agent *intent integrity* reveals failure modes in API calling under evolving toolkits (*IntenTest*) [40]. Large-scale API-agent benchmarks (*ShortcutsBench*) expose gaps in API selection, parameterization, and user-input requests [155].

System/GUI testing and multi-user scenarios. Autonomous agents drive semantic, intent-driven mobile GUI testing beyond coverage proxies (e.g., *DroidAgent*) [197]. Multi-agent collaboration surfaces deep web-app states and numerous failures (*Temac*) [87]. For multi-user features at scale, agent teams coordinate devices and roles, showing strong action similarity and defect discovery in industrial settings [38, 39]. Independent evaluations suggest how to benchmark autonomous web test agents and quantify limitations [24].

Repository-scale test execution and safety. ExecutionAgent prepares and runs project tests across diverse ecosystems, substantially improving cross-project executability [13]. Safety-focused benchmarks (e.g., *RedCode*) reveal that code agents can still execute or generate risky operations, calling for stricter defenses and evaluations [45]. Tool documentation itself can be a failure source; *ToolFuzz* systematically finds underspecifications in natural-language tool descriptions used by agents [109].

Table 3. Compact map of automated testing subareas to representative agentic systems/benchmarks.

Subarea	Representative works
Unit & oracles	CANDOR [190], TestForge [60], AdaCoder [221]
API/service	AutoRestTest [71], LogiAgent [208], IntenTest [40]
GUI/system	DroidAgent [197], Temac [87]
Multi-user	Agent-for-User/MAdroid [38, 39]
Repo-scale exec	ExecutionAgent [13]
Safety/tooling	RedCode [45], ToolFuzz [109]

What works—concise patterns.

- **Verify while you generate**: integrate build/run/coverage/fuzzing and domain oracles in the loop [45, 60, 87].
- **Role structure + memory**: planner/coder/tester/critic schemas with trajectory memory stabilize iteration [123, 221].
- **Benchmark beyond unit scope**: augment insufficient tests and emphasize repository-/system-level fidelity [13, 198].

Limitations and open challenges. (*L1*) **Evaluation fidelity**: unit-level gains do not guarantee system-level robustness [198]. (*L2*) **Safety at the interface**: agents remain vulnerable to risky execution and underspecified tools [45, 109]. (*L3*) **Intent drift**: API/tool evolution induces subtle intent misalignments despite high-level success (*IntenTest*) [40].

Actionable guidance. Close the loop early (execution and oracles); *encode roles and memory* for stable iteration; *stress-test safety and intent* with adversarial and evolving-tool scenarios; and *prefer repo-/system-scale* evaluations for external validity [13, 45, 174].

4.5 Industrial Applications and Domain–Specific Solutions

Scope and motivation. Domain-grounded agents are moving from lab prototypes to operational pipelines in *manufacturing*, *industrial control*, and *cloud operations*. These settings demand strong integration with physical or production systems, auditable plans, and safety/SLAs [156, 176, 185].

Smart manufacturing and industrial automation. LLM agents paired with *digital twins* orchestrate skills and low-level controls for flexible production, handling previously undefined tasks [185]. For safety-critical controllers, *Agents4PLC* couples code generation with verification against formal specs to yield verifiable PLC programs [94]. Industrial scene construction (*SceneGenAgent*) translates declarative layouts into executable C# with iterative verification, improving quantitative conformance [184].

Cloud systems, AIOps, and SLA-aware orchestration. Tool-augmented autonomous RCA agents (e.g., *RCAgent*) outperform rule-based and ReAct baselines in predicting root causes, remedies, and evidence, and are deployed in production incident workflows [176]. Empirical studies of retrieval-equipped agents on Microsoft production incidents outline practical considerations for tool access and data integration [146]. Vision and framework papers (*AIOpsLab*, *AgentOps*) argue for fault injection, workload generation, and standardized interfaces to evaluate agentic resilience [156]. Complementarily, *SLA-aware multi-agent RAG* exposes systems knobs (quality, cost, latency) for dynamic reconfiguration to meet SLOs under changing query intents and budgets [57].

What works—concise patterns.

- **Domain-grounded control loops**: pair agents with digital twins, formal specs, and controllers for verifiable actions [94, 185].
- **Production observability**: instrument telemetry/logs/metrics for tool-augmented diagnosis and learning [146, 176].
- **SLA-aware orchestration**: expose and tune quality/cost/latency trade-offs in multi-agent pipelines [57].

Limitations and open challenges. (*I1*) **Safety and governance**: aligning autonomy with industrial safety cases and access control remains nontrivial [94]. (*I2*) **Integration cost**: coupling agents with legacy controllers and cloud tooling requires robust adapters and schemas [146, 156]. (*I3*) **Evaluation realism**: few works report long-horizon reliability under realistic failure and workload regimes; fault-injection testbeds and standardized metrics are still evolving [156].

Actionable guidance. Design for verifiability first (digital twins, specs, audit trails); *co-design with operations* (telemetry, tool access, privacy); and *treat SLAs as first-class* by making cost/latency/quality tunable in orchestration policies [57, 176, 185].

4.6 Educational and Research Applications

Scope and significance. This stream covers (i) programming education and tutoring, (ii) research assistance and methodology automation, and (iii) evaluation ecosystems and benchmarks for studying agentic systems. Across these contexts, agents are most effective when they combine tool grounding, explicit role schemas, and transparent planning to provide auditable learning and research workflows [51, 80, 88].

Programming education and tutoring. Teachable-agent paradigms deliver measurable learning gains by turning students into teachers who diagnose LLM code errors and reflect on hypotheses [97]. Constructionist interfaces (e.g., ChatLogo) scaffold agent-based modeling via hybrid natural/programming languages for novice learners [18]. Multimedia pedagogical agents (e.g., VTutor) extend beyond text to animated feedback and multi-modal interaction, aiming to raise engagement and receptivity [17]. Beyond tutoring, communication-centric benchmarks such as HumanEvalComm quantify the ability to ask clarifying questions—a competency closely tied to expert developer behavior [181].

Research assistance and methodology automation. Agents increasingly support research workflows: multi-agent qualitative analysis and SLR automation reduce effort across search, screening, and synthesis while preserving researcher control [138, 150]. At the method–implementation boundary, ResearchCodeAgent codifies algorithmic procedures into executable pipelines with iterative planning and memory [41]. Two emerging lines target scientific reproducibility: LMR-BENCH for code reproduction in language modeling research [193] and SciReplicate-Bench for paper-to-code algorithmic replication with reasoning-graph evaluation [186]. Domain deployments such as MARAUS (multi-agent RAG for university admissions counseling) illustrate operational constraints and cost–latency trade-offs in real educational settings [115].

Evaluation ecosystems and agent study tooling. Agent evaluation is moving from function-level coding toward interactive, environment-driven tasks: CoSQA+ couples code search with test-driven agent annotation [43]; ShortcutsBench probes API-based agents' planning, parameterization, and user-input handling [155]; AppWorld and BrowserGym provide controllable multi-app/web environments for rich, programmatically scored tasks [25, 165]. Visual analytics such as SeaView help researchers inspect long trajectories and compare experimental runs [14]. Large-scale audits of agent-generated patches on real GitHub issues highlight heterogeneous behaviors and benchmark

blind spots [20], while agentless baselines remind us that simpler, interpretable pipelines can be competitive and cheaper in some settings [183].

What works—concise patterns.

- **Teach-by-diagnosis**: framing learners as critics/teachers improves debugging skills and transfer [97].
- **Multi-modal scaffolding**: animated and hybrid-language interfaces broaden accessibility for novices [17, 18].
- **Evaluation in context**: use interactive, tool-rich benchmarks and trajectory analytics to assess real competencies [14, 25, 165].

Limitations and open challenges. (*E1*) **External validity**: gains on small tasks do not always carry to open-ended learning or research reproduction [20, 193]. (*E2*) **Communication competence**: agents still under-ask clarifying questions in ambiguous educational problems [181]. (*E3*) **Cost and governance**: sustained classroom or lab deployments must manage budgets, privacy, and auditability [115].

Actionable guidance. Instrument *communication* as a first-class metric; adopt *interactive, test-driven* evaluation; and favor *transparent role schemas and memory* for teachability and auditability [43, 80, 181].

4.7 Development Process Automation

Scope and significance. This area targets end-to-end development workflows—requirements, planning, coding, review, testing, integration, and release—by *automating process execution* and *instrumenting evaluation* around real repositories and tools [88, 174].

Process emulation and orchestration. Agent teams that emulate software process models (e.g., Waterfall, TDD, Scrum) with planner/architect/developer/tester roles improve consistency and traceability across stages [83]. Repo-level systems (CodeAgent) integrate search, symbol navigation, and testing to act on complex codebases rather than isolated snippets [206]. At the same time, agentless pipelines show that carefully engineered three-phase workflows (localize–repair–validate) can match or beat complex agents on cost and accuracy for certain tasks [183]. Large-scale audits of agent patches stress that benchmark tests can miss semantic gaps and that issue complexity strongly influences outcomes [20].

From workflows to operations. Process automation increasingly couples with AIOps to handle incidents and reliability at scale; AIOpsLab proposes a holistic evaluation environment with fault injection, workload generation, and telemetry for agent assessment [19]. Evaluation-driven development further reframes agent building as a continuous loop that integrates online/offline testing, human/AI feedback, and architecture refinement [182]. To study planning and tool use beyond coding, API-centric and multi-app ecosystems (ShortcutsBench, AppWorld, BrowserGym) expose agents to realistic orchestration and parameter-filling challenges [25, 155, 165].

Datasets and benchmarks for process fidelity. New corpora and harnesses emphasize repository- and project-level fidelity and diversity: SWE-rebench provides continuous, decontaminated tasks for training and evaluation [10]; SWE-PolyBench expands to multiple languages and task types with execution-based assessment [141]; ProjectEval simulates user interaction to judge project-level generation [89]. Process-focused surveys highlight a systemic imbalance toward development over upstream phases, motivating broader coverage and cross-phase metrics [170].

What works—concise patterns.

- **Role schemas + tool grounding**: emulate process roles and wire agents (or agentless stages) to build/test tooling for verifiable progress [83, 183, 206].
- **Evaluation-driven loops**: unify online telemetry, offline testbeds, and human/AI feedback to iteratively refine pipelines [19, 182].
- **Benchmark breadth**: prefer repository-/project-level, multi-language, and API/multi-app tasks to capture orchestration competence [10, 141, 155, 165].

Limitations and open challenges. (*P1*) **Generalization across stacks**: agents often struggle to transfer workflows across languages, frameworks, and ops environments [19, 141]. (*P2*) **Benchmark blind spots**: unit tests and issue statements can mask semantic differences from gold fixes; project-level user simulation remains nascent [20, 89]. (*P3*) **Process coverage**: requirements/design remain under-evaluated relative to coding, limiting evidence for truly end-to-end automation [170]. (*P4*) **Trajectory observability**: long, tool-rich runs are hard to inspect without specialized analytics [14].

Actionable guidance. Design for verification first (tests, telemetry, replayable runs), adopt *evaluation-driven development* to close the loop continuously, and expand *benchmark breadth* toward project-/ops-scale tasks with multi-language, multi-app, and API orchestration to expose real process competence [10, 19, 182].

4.8 Human–AI Interaction and Collaboration

Scope and significance. This strand examines how agentic systems collaborate with developers and stakeholders through conversational interfaces, role-structured workflows, and human-in-the-loop (HITL) protocols. Two cross-cutting levers consistently emerge: (*i*) *communication competence*—the ability to identify ambiguity and ask clarifying questions—and (*ii*) *trajectory observability*—logging, replay, and comparison of long action chains for audit and learning [14, 181].

Interaction modalities and role schemas. Role-structured dialogues (planner/architect/developer/tester) support negotiated plans and reduce prompt drift; educational variants show how hybrid natural/programming languages and multimodal feedback scaffold novice learning and reflection [17, 18]. In professional workflows, context-aware copilots that surface relevant artefacts (files, tests, symbols) during navigation shorten clarification turns and raise perceived usefulness [206].

HITL protocols and observability. HITL schemes that reward "ask-before-act," acknowledgement steps, and post-hoc critique correlate with fewer erroneous edits and higher task success on coding-style tasks [181]. For long-horizon activities, trajectory analytics (visual diff/replay, side-by-side run comparison, event timelines) enable diagnosis and reproducibility, supporting both iterative engineering and instruction [14].

Table 4. Key human–AI interaction affordances and reported effects.

Affordance	Implementation pattern	Reported effect
Clarifying questions	Ask-before-act, acknowledgement gates	Fewer erroneous edits; higher task success [181]
Trajectory observability	Log/replay, run comparison, timelines	Easier diagnosis; reproducibility; pedagogy [14]
Context-aware retrieval	Surface artefacts during navigation	Shorter clarification; higher utility [206]
Multimodal scaffolding	Animation, hybrid language interfaces	Engagement; novice accessibility [17, 18]

What works—consolidated patterns.

- **Negotiate plans, not answers**: use role schemas and stepwise commitments to maintain alignment during complex tasks [18].
- **Instrument communication**: measure clarifying-question rates and acknowledgement efficacy; make these first-class metrics [181].
- **Make trajectories auditable**: require replayable traces and comparison views in evaluation and practice [14].

Limitations and open challenges. (*H1*) Generalisable dialogue policies that robustly elicit missing constraints remain open. (*H2*) Over-automation risks: without visibility/checkpoints, users can over-trust fluent but brittle agents. (*H3*) Cost/privacy: sustained logging and replay demand governance and budgets [14].

Actionable guidance. Adopt *ask-then-act* policies with explicit acknowledgement gates; pair copilots with repository/IDE context signals; and standardise trajectory capture for trust, pedagogy, and continuous improvement [14, 181].

4.9 System Integration and Tool Development

Scope and significance. As agent research moves from toy sandboxes toward production-grade stacks, integration concerns dominate: discovering and composing external APIs, embedding in IDE/no-code environments, and bootstrapping heterogeneous tool-chains with safe, observable execution [165, 173].

API orchestration and service composition. Multi-app ecosystems expose long-horizon planning: tool discovery, parameterisation, pre/post-condition checks, and effect verification. Controlled environments such as AppWorld repeatedly surface planning and parameterisation bottlenecks that do not appear in single-API tasks [165].

IDE/no-code embedding and execution sandboxes. Visual authoring canvases (AI2Apps, AutoGen Studio) reduce friction in building/debugging recursive workflows, while open execution platforms provide shell/browser/runners with replayable traces and guard rails (resource/time limits, isolation) for deep inspection and safer trials [29, 124, 173].

Tool-chain automation and reproducibility. Automated build/installation frameworks mine documentation, resolve dependencies, and configure compilers/runtimes across repositories; replayable runs and artefact capture improve executability and enable regression investigation [173].

Table 5. Integration strands and representative platforms/signals.

Strand	Representative platform	Primary evaluation signals
API orchestration	AppWorld [165]	Tool selection accuracy; parameter correctness; effect checks
Visual/no-code IDE	AI2Apps; AutoGen Studio [29, 124]	Build/debug latency; authoring errors; workflow reuse
Execution/sandbox	OpenHands [173]	Executability; safety incidents; replay fidelity

What works—consolidated patterns.

- **Standardise interfaces early**: typed messages and discovery protocols reduce glue-code brittleness.
- **Observe everything**: sandboxed execution with rich logs/replays is indispensable for debugging long trajectories [173].

- **Close the install gap**: treat build/install as first-class; preflight dependency checks before planning actions.

Limitations and open challenges. (*S1*) Bootstrap brittleness in environment setup and third-party tooling. (*S2*) Long-horizon composition across APIs remains a key failure mode in multi-app tasks [165]. (*S3*) Security/governance for open shells and dynamic tools requires isolation, least-privilege defaults, and auditable traces [173].

Actionable guidance. Invest in *sandboxed runners with replay*, *visual workflow authoring* for rapid iteration, and *early automated tool-chain checks* to cut time-to-first-action [29, 124, 173].

4.10 Legacy System Modernization and Documentation

Scope and significance. Long-lived systems concentrate technical debt: obsolete languages, sparse documentation, and multi-decade dependency tangles. Agentic pipelines address these through (*i*) deep code comprehension over large histories, (*ii*) automated documentation and knowledge capture, and (*iii*) staged transformation/migration with verification.

Deep comprehension with repository context. Repository-scale "deep-research" agents chain search, summarisation, and patch synthesis across code, tests, issues, and commit history; empirical reports show higher crash-resolution rates than standard baselines, emphasising the importance of mining both program semantics and historical signals [160]. Trajectory analytics further support onboarding and long-horizon refactoring by making prior decisions queriable and auditable [14].

Documentation and knowledge preservation. Multi-agent documentation planners improve completeness over single-agent baselines by ordering traversal (module→file→API), separating writer/reviewer roles, and validating outputs against code/tests. Persisting these artefacts with replayable traces yields institutional memory that accelerates future maintenance and reduces rediscovery costs [14].

Modernization and migration. Staged pipelines decompose upgrades/ports into explicit phases (inventory→translation→dependency updates→regression tests), with pass/fail gates and rollbacks. Although fully behaviour-preserving translation at scale remains challenging, project tests and differential traces provide practical oracles to bound regressions [160].

What works—consolidated patterns.

- **Plan–verify loops**: structure traversal/order, include reviewer/validator roles, and verify with project tests.
- **Exploit history**: treat commits/issues/docs as first-class context for comprehension and refactoring [160].
- **Retain trajectories**: log/replay decisions and transformations to create auditable, reusable knowledge [14].

Limitations and open challenges. (*L1*) **Semantic preservation**: ensuring behaviour-equivalent translations across language/ABI changes is difficult. (*L2*) **Evaluation fidelity**: small exemplars understate mainframe/embedded constraints; repo-scale, domain-aware metrics are needed. (*L3*) **Tool coverage**: legacy ecosystems expose underspecified/missing tools; bridging them demands robust adapters and fallbacks.

Actionable guidance. Design *continuous modernisation loops* that alternate comprehension, translation, and documentation under shared memory and verification; prioritise repository-scale

evaluations with domain-specific oracles; and mandate trajectory/log retention for audit and rollback [14, 160]. regime.

RQ2 – Summary

(1) **Universal building blocks**: orchestration engines, RAG grounding, transparent CoT/ToT planners, and tool/runners form the common substrate across all task families.
(2) **Multi-agent leverage**: role schemas, hierarchical/supervisory control, and debate/consensus on standard buses improve robustness and accuracy by 5–28 pp under budget-aware cost/latency orchestration.
(3) **Verification in the loop**: hybrid static/dynamic analysis, tests/fuzzing, and critic gates turn generation into a continuous optimisation cycle and expose failures at repository scale.
(4) **Human-centred interaction & observability**: ask-before-act policies, context-aware copilots, and replayable trajectories reduce clarification turns and surface rationales for audit and learning.
(5) **Integration, domain, and legacy**: API/multi-app orchestration, visual/no-code IDEs, and sandboxed, reproducible runners enable deployment; domain-grounded and modernisation pipelines meet safety/compliance yet highlight integration debt.

5 RQ3: WHAT ARE THE MAIN CHALLENGES AND FUTURE RESEARCH DIRECTIONS IN APPLYING INTELLIGENT AGENTS TO SE?

Building on the conceptual landscape and scope (RQ1) and the enabling technologies and patterns (RQ2), this section interrogates the barriers that currently constrain Intelligent Agents in software engineering and the opportunities that can unlock their next stage of impact. We distill the core limitations affecting effectiveness, scalability, and trustworthiness in real-world settings and outline research directions aimed at overcoming them.

Transitioning from prototype demonstrations to production-grade systems is a socio-technical transformation as much as a technical one. Our systematic analysis of 215 studies indicates that strong results in controlled settings often fail to carry over to deployment environments, underscoring the need for coordinated advances in evaluation, efficiency, safety, human factors, integration, and governance.

The six challenge categories synthesized in Figure 11 reflect cross-cutting, system-level barriers rather than idiosyncrasies of particular implementations: *Lack of Rigorous Evaluation and Benchmarks* (difficulty assessing end-to-end workflows, multi-agent interaction, and human factors); *Scalability and Efficiency Constraints* (computational and economic feasibility at repository and organization scale); *Safety and Trustworthiness* (reliable, explainable, and accountable operation in high-stakes contexts); *Human–Agent Interaction Limitations* (effective, mixed-initiative collaboration without cognitive overload); *Integration and Deployment Complexities* (fit with existing toolchains, CI/CD, and ops practices); and *Data Privacy and Governance* (policy, regulatory, and ownership concerns around proprietary code and sensitive telemetry).

The roadmap situates the field in a phase of fragmented point solutions and calls for a focused research push during 2025–2027 to align advances across all dimensions. Expected gains from successfully addressing these challenges include broader, lifecycle-complete benchmarks, substantial cost reductions, stronger formal safety assurances, seamless human collaboration, cross-domain adaptability, and production-ready integration.

In sum, progress toward mature, widely adopted agentic systems requires *coordinated* advances across technical capabilities, evaluation methodology, safety frameworks, human-centred design,

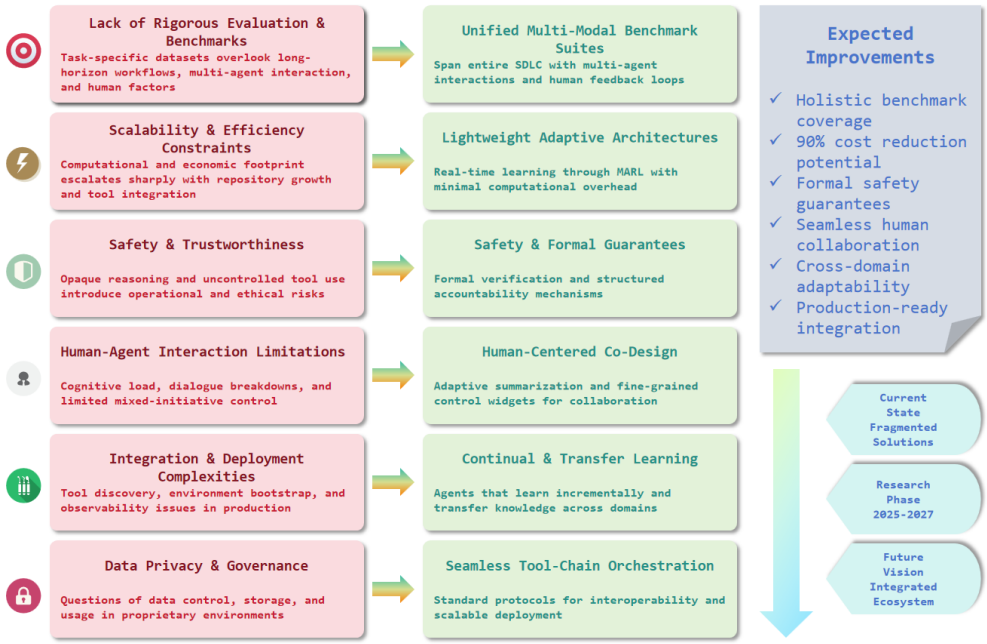


Fig. 11. Challenges and Future Research Directions Roadmap for Intelligent Agents in Software Engineering. The diagram maps six primary challenge categories (left) to corresponding research directions (center) and expected improvements (right), charting a progression from fragmented solutions toward an integrated ecosystem (2025–2027).

integration tooling, and governance. The following *Key Challenges* analyze each category in turn, articulate specific barriers, review current mitigation attempts, and motivate concrete research directions grounded in the roadmap of Figure 11.

5.1 Key Challenges

Our analysis identifies six primary challenges that constrain the transition of Intelligent Agents from promising prototypes to robust, production-grade systems in SE: (*i*) lack of rigorous, system-level evaluation; (*ii*) scalability and efficiency under repository- and organization-scale workloads; (*iii*) safety, explainability, and operational trust; (*iv*) limitations in human–agent interaction (HAI) and mixed-initiative control; (*v*) integration and deployment in heterogeneous toolchains; and (*vi*) privacy, ownership, and governance. These challenges recur across tasks and architectures, and they often interact—for example, weak observability hinders both safety auditing and reproducible evaluation. To improve readability and avoid page overflow, we split the overview into two tables: Table 6 (Categories 1–3) and Table 7 (Categories 4–6).

5.1.1 Lack of Rigorous Evaluation and Benchmarks. Current evaluations are dominated by *narrow* tasks, single-agent settings, and short horizons, which under-represent the complexity of repository-scale work and socio-technical feedback loops. As in Tables 6 and 7, CodeAgentBench exercises repository navigation but does not quantify planning quality; HumanEvalComm probes communication competence but lacks multi-file and multi-agent coordination [181, 206]. Meta-analyses further reveal reporting gaps (e.g., only 27% track memory overhead), limited cost/latency

Table 6. Overview of challenges and representative solutions in Intelligent Agent systems (Part I: Categories 1–3)

Challenge Category	Framework/Study	Approach	Key Metric	Result/Limitation
1. Evaluation and Benchmarks				
	HumanEvalComm [181]	Unit coding + comm. probes	Dialogue rate	Lacks repository context
	CodeAgentBench [206]	Repository-level coding	Tool navigation	No explicit planning metric
	SWE-Bench Lite [8]	Issue fixing	Pass rate	Ignores cost/latency
	Meta-study [174]	Survey analysis	Coverage metrics	27% report memory only
2. Scalability and Efficiency				
	SoA [59]	Agent splitting	Context/agent	Fixed scaling assumptions
	AI2Apps [124]	Selective invocation	API calls	~80% reduction
	MCP-Zero [36]	Active tool discovery	Prompt tokens	~98% reduction
	RepairAgent [11]	FSM gating	Cost per bug	$0.14/bug (task-specific)
3. Safety and Trustworthiness				
	Concept-Guided [42]	Safety ontology	Hazard analysis	Structured, inspectable reports
	Watson [142]	Trace reconstruction	Pass@1	+12.3 pp with observability
	RCAgent [176]	Self-consistency filtering	RCA accuracy	+13 pp; ops focus
	DepsRAG [5]	Dependency reasoning	Supply chain	No formal guarantees

accounting, and scarce failure taxonomies [174]. *Implication.* The field needs end-to-end, *trajectory-aware* benchmarks that capture agent plans, tool invocations, and human interventions across the SDLC, with standardized logs, cost/quality trade-offs, and cross-agent reproducibility protocols.

5.1.2 Scalability and Efficiency Constraints. Agent pipelines can incur steep token and wall-clock costs at repository scope; for instance, static-repair agents may consume up to 270k tokens per bug [11]. Selective invocation and active tool discovery reduce waste (e.g., ~80–98% fewer API descriptions or calls), and agent splitting maintains context budgets [36, 59, 124]. Yet scaling remains brittle: fixed resource policies fail under workload skew, and monolithic models overfit token budgets. *Implication.* Practical systems require *adaptive* routing (task-aware model/tool selection), streaming/RAG caches, budgeted planning, and failure-aware retries—*with* cost/latency reported as first-class metrics alongside accuracy [49].

5.1.3 Safety, Explainability, and Trustworthiness. Opaque reasoning, tool misfires, and under-specified permissions create operational risk. Safety ontologies structure hazard identification and evidence trails [42]; cognitive observability (trace reconstruction, plan logging) improves headline metrics such as Pass@1 by 12.3 pp [142]; self-consistency filters raise root-cause accuracy by 13 pp in cloud incidents [176]. However, guarantees are typically *post hoc* and domain-limited (e.g., supply-chain reasoning without formal proofs [5]). *Implication.* Standardized safety audits should

Table 7. Overview of challenges and representative solutions in Intelligent Agent systems (Part II: Categories 4–6)

Challenge Category	Framework/Study	Approach	Key Metric	Result/Limitation
4. Human–Agent Interaction				
	ChatDev [132]	Multi-role dialogue	User control	59% report loss of agency
	HumanEvalComm [181]	Clarifying questions	Query rate	34% in ambiguous cases
	HypoCompass [97]	Teach-back loops	Learning efficiency	×4 improvement
	Industrial survey [76]	Field study	Abandonment rate	42% after 3 turns
5. Integration and Deployment				
	Installamatic [110]	PyPI env setup	Build success	55% success
	CompileAgent [52]	C/C++ build skills	Build success	71% success
	CSR-Agents [187]	Research repos	Deploy success	53% success
	SetupBench [7]	Cross-agent tests	Reproducibility	38% cross-agent
6. Privacy and Governance				
	PolicyGuard [95]	Role-based access	Data protection	Principled walls
	Concept-Guided [42]	Ontology tagging	Provenance	Audit trails
	RCAgent [176]	On-premise ops	Data leakage	Zero external calls
	Personal Agents [81]	Threat taxonomy	Risk analysis	Design guidelines

pair *ex ante* policy checks (capability/permission bounds) with *in situ* monitors (trace, anomaly, rollback) and *ex post* certification pipelines for dependable deployment.

5.1.4 Human–Agent Interaction Limitations. Field evidence indicates cognitive overload and weak mixed-initiative control: agents ask clarifying questions in only 34% of ambiguous cases; developers report loss of agency in long sessions and abandon after a few unresolved turns [76, 132, 181]. Conversely, teach-back loops substantially improve learning efficiency [97]. *Implication.* Interfaces should emphasize progressive disclosure (why/what-next), editable plans, and granular *stop/undo/branch* controls, with interaction metrics (clarification rate, takeover latency) treated as optimization targets rather than afterthoughts.

5.1.5 Integration and Deployment Complexities. Productionization remains fragile across ecosystems: environment setup and build success rates vary widely (55–71%), deployment on research repos shows modest success (53%), and cross-agent reproducibility is low (38%) [7, 52, 110, 187]. Tool discovery and orchestration are still manual in many pipelines, despite promising automation (e.g., CodeNav, MCP-Zero). *Implication.* Robust deployment calls for standardized orchestration schemas (tools, permissions, artifacts), end-to-end tracing APIs (plan→tool→artifact lineage), and cross-agent CI that replays logs to verify results deterministically.

5.1.6 Data Privacy, Ownership, and Governance. Agent access to proprietary code and telemetry raises leakage and accountability concerns [81]. Dependency- and supply-chain reasoning surfaces transitive risks [5]. Emerging controls—role-based access, provenance tagging, on-premise execution—mitigate exposure but are rarely unified into an auditable governance stack [42, 95, 176]. *Implication.* Organizations need defense-in-depth: principle-of-least-privilege policies, data minimization and redaction at ingestion, immutable provenance logs, and periodic governance reviews aligned to regulatory and contractual obligations.

5.2 Future Research Directions

5.2.1 Unified, Multi-Modal Benchmark Suites. The need for unified, multi-modal benchmark suites to evaluate LLM-based agents in software engineering tasks is critical to advancing the field. Traditional benchmarks, such as *HumanEval* for code generation [181], are primarily focused on isolated tasks such as coding or bug-fixing. However, software engineering is a multi-faceted process, involving continuous interaction between multiple stages such as requirements engineering, system design, testing, and deployment. To capture the full complexity of these workflows, future research should prioritize the development of benchmark suites that span the entire software development lifecycle (SDLC). This would ensure that agents are tested not just on isolated coding tasks but also on their ability to work collaboratively across various stages, managing complex dependencies and interactions in a real-world context.

For example, a multi-modal benchmark suite could integrate various components such as code generation, testing, and refactoring in a manner similar to the *SWE-Bench Lite* framework [8], but with the added focus on multi-agent collaboration and human feedback. The integration of these diverse task categories into a unified suite would require developing metrics that are comprehensive enough to cover a broad range of tasks and that emphasize both task completion and collaboration between agents. Moreover, incorporating real-world software projects, as demonstrated by [206], could provide more reliable and actionable performance data, improving the evaluation of multi-agent systems at scale.

Additionally, as the industry moves towards more complex AI-driven workflows, it is crucial that benchmarks go beyond simple accuracy metrics and incorporate other dimensions such as resource efficiency, model robustness, and the ability to adapt to unforeseen changes in the development environment. For instance, *RepoAudit* [46] provides insights into repository-level code audits, but lacks the depth needed to measure long-term agent interactions and system-wide performance. Thus, creating benchmarks that evaluate agents across these diverse domains and conditions will be a fundamental step towards the widespread adoption and trust of AI in software engineering.

5.2.2 Lightweight and Adaptive Agent Architectures. A critical challenge in applying Intelligent Agents to software engineering is the development of lightweight and adaptive agent architectures that can scale efficiently while maintaining high performance. Current agent-based frameworks often require significant computational resources, limiting their applicability in resource-constrained environments, such as embedded systems or mobile applications. Therefore, one of the primary areas for future research is the design of lightweight agents that can handle complex tasks with minimal computational overhead. Recent studies, such as those by [124], have explored visual IDEs that optimize computational efficiency by selectively invoking only necessary tools, reducing both token usage and latency. These approaches represent the first steps toward more efficient and adaptive systems that can function within resource-limited environments.

Moreover, adaptive agent architectures are necessary to ensure that the agents can modify their behavior in response to dynamic software development conditions. This requires agents that can learn from experience and continuously refine their strategies. Research in this area could build on existing frameworks like *SoA* [59], which automatically adjusts the number of agents and their tasks based on the workload. An adaptive framework that integrates learning algorithms with real-time feedback loops could further enhance agent efficiency. This would be particularly valuable in contexts where software environments are continuously evolving, such as in agile development teams or dynamic CI/CD pipelines.

To achieve this, future agent architectures should incorporate real-time learning and adaptation, potentially through multi-agent reinforcement learning (MARL) or similar methodologies. These architectures must also be flexible enough to be integrated into existing development workflows

without requiring significant reconfiguration, ensuring that agents can be easily deployed across different environments and projects. An adaptive system capable of autonomously adjusting its approach to tasks based on evolving project requirements would significantly improve both the scalability and efficiency of LLM-based agents in software engineering.

5.2.3 Safety, Accountability, and Formal Guarantees. As the deployment of LLM-based agents in software engineering environments increases, ensuring their safety and accountability becomes a paramount concern. These agents, which are increasingly capable of making autonomous decisions, must be designed to operate within clearly defined safety constraints, ensuring that they do not cause harm or make decisions that lead to undesirable outcomes. The safety of these agents is not just a technical concern but a critical ethical issue, especially in mission-critical systems like aerospace, healthcare, and financial software, where the consequences of an error can be severe. Research in this area should focus on embedding safety features into LLM-driven systems, ensuring that agents can be trusted to operate within predefined safety boundaries. For instance, [42] has proposed integrating safety ontologies into agents' reasoning processes, allowing agents to make decisions based on a structured, transparent understanding of potential hazards.

Formal verification techniques could also be employed to ensure that the behavior of Intelligent Agents aligns with their intended objectives, particularly in high-risk domains. Formal methods, as discussed by [95], offer a promising approach to proving the correctness of agent decision-making processes, ensuring that agents do not violate safety constraints. However, the application of formal methods to LLM-based agents remains an open problem, particularly when dealing with the complexity and scale of real-world software engineering tasks.

In addition to safety, accountability is another essential element for LLM-driven systems. It is important to ensure that there is a clear chain of responsibility for the actions taken by agents, especially when those actions result in negative outcomes. Future research should explore techniques for auditing agent decisions and ensuring that there is transparency in how decisions are made. This could involve developing agent models that store a clear record of their decision-making process, enabling post-mortem analysis when issues arise. Furthermore, establishing regulatory frameworks and guidelines for the ethical deployment of these agents, including how to address potential biases in agent decision-making, will be crucial for fostering public trust in AI systems.

The intersection of safety and accountability will define the next generation of AI-driven software engineering tools. Formal safety frameworks, when combined with robust accountability mechanisms, will ensure that agents can be trusted to make decisions in complex, high-stakes environments. Research in this area will also need to address the challenge of integrating these safety and accountability features into the agent's architecture without compromising performance or scalability. Achieving this balance is a critical research direction that will drive the broader adoption of LLM-based agents in real-world software engineering tasks.

5.2.4 Human-Centered Co-Design and Interaction Protocols. Human-centered co-design and interaction protocols are essential to ensuring that Intelligent Agents align well with user needs and expectations in real-world software engineering applications. While much of the current research focuses on the technical capabilities of these agents, a growing body of work emphasizes the importance of user-agent collaboration. In particular, research into agent-based interfaces highlights the need for intuitive communication and decision-making support for human engineers, thereby fostering a seamless integration between machine intelligence and human expertise. Human-centered design is also pivotal in ensuring that agents operate within ethical boundaries, respecting human decision-making autonomy and addressing biases present in the agent's training data.

For example, [97] explores how intelligent tutoring systems can enhance the learning experience for software developers by offering personalized hints and guidance. Similarly, [138] discusses the

role of conversational agents in supporting software development tasks by engaging in natural dialogue with users to refine project requirements and development goals. These systems depend heavily on robust interaction protocols that are adaptable and context-aware, making them crucial for bridging the gap between human needs and machine intelligence.

Looking forward, future research must explore methods to optimize human-agent collaboration, including developing protocols that account for varying levels of agent autonomy and user interaction preferences. Establishing effective feedback loops, where the agent can adapt its responses based on user input, is another promising avenue of research. These advancements will ensure that Intelligent Agents remain supportive tools rather than disruptive elements within software engineering workflows.

5.2.5 Continual, Lifelong, and Transfer Learning. As software systems evolve, the need for Intelligent Agents to learn continually and transfer knowledge across tasks has become increasingly important. Traditional machine learning models are often trained on static datasets and cannot adapt to changes in the environment without retraining, which can be resource-intensive and impractical. In contrast, continual, lifelong, and transfer learning techniques aim to equip agents with the ability to learn incrementally from new data and transfer knowledge from one domain to another, enabling them to improve over time and generalize across different tasks.

For example, [190] discusses how multi-agent systems, by leveraging lifelong learning principles, can enhance their ability to generate more accurate solutions by learning from each interaction. Additionally, [44] highlights the challenges and opportunities of applying transfer learning to the development of autonomous programming agents that can adapt to various programming environments and languages without requiring full retraining. These approaches are essential for real-world applications where agents must continually learn from new codebases, user feedback, and evolving software requirements.

The research community must focus on developing efficient techniques for agent adaptation that minimize the costs associated with continual learning while maintaining performance. Transfer learning also offers a powerful approach for improving agent versatility across multiple domains, allowing agents to leverage knowledge gained from one task to enhance their performance on another. Bridging these learning paradigms will result in more intelligent and adaptable systems capable of scaling with the increasing complexity of software engineering tasks.

5.2.6 Seamless Integration and Tool-Chain Orchestration. Seamless integration and tool-chain orchestration are foundational for ensuring that Intelligent Agents can be effectively deployed in real-world software engineering environments. In modern software development, numerous tools are used for tasks such as code analysis, testing, deployment, and debugging. The challenge lies in integrating these tools into a cohesive workflow that maximizes efficiency and reduces manual intervention. Intelligent Agents play a critical role in orchestrating these tool chains, automating repetitive tasks, and optimizing the development process.

For instance, [176] explores how multi-agent systems can integrate with cloud infrastructure to automate root-cause analysis, which is an essential task in incident management. Similarly, [124] discusses how AI-based tools can assist developers by integrating various code-generation and debugging tools within a unified development environment, enabling real-time feedback and reducing the time spent on debugging and testing. These advances highlight the importance of designing systems where multiple tools and agents work together to facilitate the entire software development lifecycle.

Moving forward, research should focus on developing standard protocols for tool-chain integration, ensuring that agents can seamlessly interact with existing development environments and workflows. Interoperability between different tools, platforms, and agents must be prioritized,

with an emphasis on minimizing friction for developers. Additionally, orchestration frameworks must support scalability, enabling the deployment of complex, multi-agent systems across large-scale software projects. Through seamless integration, Intelligent Agents can transform software development by automating tasks, improving productivity, and ensuring higher-quality outputs.

RQ3 – Summary

(1) **Evaluation & benchmarks**: Current tests are narrow and short-horizon; the field needs trajectory-aware, lifecycle-complete, multi-agent+HAI benchmarks with standardised logs, cost/latency/memory/tool-use reporting, and cross-agent reproducibility.
(2) **Scalability & efficiency**: Repository-scale pipelines are token- and latency-heavy; adopt adaptive routing (model/tool selection), selective invocation, streaming/RAG caches, budgeted planning, and failure-aware retries.
(3) **Safety & trustworthiness**: Opaque plans and risky tool use impede deployment; combine ex-ante capability/permission bounds with in-situ monitors (traces/anomaly/rollback) and ex-post certification; pursue formal guarantees where feasible.
(4) **Human–agent interaction & observability**: Mixed-initiative protocols are immature; enforce ask-before-act, editable plans, progressive disclosure, and replayable trajectories; treat clarification rate and takeover latency as first-class metrics.
(5) **Integration, deployment, privacy & governance**: Environment setup and CI/CD reproducibility remain brittle; standardise orchestration schemas and sandboxed, auditable runners; apply least-privilege access, data minimisation, and provenance.
(6) **Roadmap (2025–2027)**: build unified multi-modal SDLC benchmarks; design lightweight, adaptive architectures; integrate safety-by-design and governance; and raise cross-agent reproducibility on real projects.

6 DISCUSSION

This review shows that, despite heterogeneous goals and settings, effective intelligent-agent systems for SE converge on a common substrate: *orchestration engines*, *retrieval-augmented grounding*, *transparent planners* (e.g., CoT/ToT), and *deep tool integration*. These pillars reappear across all task families we analyzed and frame why certain prototypes generalize better than others (Fig. 10). At the same time, publication patterns reveal an uneven emphasis: SDLC management, multi-agent collaboration, code analysis/QA, and testing account for the majority of studies, while system integration/tooling and legacy modernization remain underrepresented. This asymmetry partially explains why strong sandbox results do not always translate into robust end-to-end performance in production contexts.

Interpretive synthesis. Two system-level inferences follow from our evidence. First, the apparent diversity of agent applications masks an architectural unity: systems that make orchestration, grounding, planning, and tool use first-class concerns deliver more stable behavior across repositories and workflows. Second, *evaluation fidelity* is now the principal bottleneck. Narrow, short-horizon tasks are poor proxies for repository- and organization-scale work where planning quality, tool-use reliability, and human–agent interaction (HAI) determine success. A trajectory-aware view—plans, tool calls, costs/latency, memory, and intervention points—should therefore become part of the evidence base, not an implementation detail.

Implications for practice. For practitioners seeking dependable adoption, design choices should prioritize: (i) **orchestration first**—define roles, artifact contracts, and progression gates before

model selection; (ii) **ground broadly, plan transparently**—pair repository-wide retrieval with auditable plans to curb drift and ease debugging; (iii) **verify while you act**—integrate tests, analysis, and critic checkpoints to convert generation into a continuous improvement loop; (iv) **instrument cost/latency and memory** as first-class KPIs; and (v) **run under sandboxed, auditable execution** with replayable traces to support diagnosis, safety, and compliance. These practices align with the four pillars above and mitigate the most frequent failure modes we observed across categories.

Implications for research. The field would benefit from shifting evidence standards from function-level accuracy toward *lifecycle-complete, trajectory-aware* evaluation. Concretely: build benchmarks that span multiple SDLC stages and include multi-agent+HAI protocols; report standardized logs (plans, tool invocations, traces) alongside accuracy, with *cost/latency/memory/tool-use success* as required metrics; adopt cross-agent reproducibility harnesses and replayable CI to compare systems fairly; and treat *safety-by-design* as integral—not post hoc—combining capability bounds, permissioning, and in-situ monitors. Finally, underexplored areas such as system integration and legacy modernization deserve targeted, repository-scale tasks to close the lab–production gap.

Positioning with respect to limitations. We intentionally reserve detailed validity threats (internal, external, construct, and conclusion) for the dedicated section to avoid duplication. In brief, the same issues that beset the literature—narrow benchmarks, small controlled environments, and incomplete system metrics—also bound our synthesis of reported effects. Our artifact release and explicit focus on trajectory observability and system-level metrics are designed to facilitate follow-on replications and longitudinal extensions.

Closing remarks. Our consolidated view is that Intelligent Agents already provide a unifying paradigm for SE automation, provided four pillars are engineered as first-class modules and assessed with system-level evidence. The roadmap that emerges is pragmatic: elevate evaluation fidelity, standardize observability and governance, and expand coverage to integration- and legacy-heavy settings. Doing so will move the community from promising point solutions toward dependable, production-grade agentic systems.

7 THREATS TO VALIDITY

We discuss validity threats following the common taxonomy of *internal*, *construct*, *conclusion*, and *external* validity, and outline mitigations built into our SLR protocol, aligned with established guidance for systematic reviews in software engineering and the broader evidence-based tradition [73, 111, 128]. Methodological details are provided in Section 2.

Internal validity. Selection and extraction bias can arise during screening and data coding. We mitigated these risks with dual-review screening (title/abstract and full text) followed by reconciliation using a standardized decision form with explicit inclusion/exclusion rationales (peer-reviewed venues only). Data extraction likewise used a standardized instrument covering bibliometrics, task categories, enabling technologies, and reported evaluation signals, and was independently performed by two reviewers; inter-rater reliability was high (Cohen's $\kappa = 0.89$) prior to consensus. Residual risks remain for borderline cases or studies with incomplete reporting; these were flagged in our logs and, where necessary, excluded from quantitative tallies while retained for qualitative synthesis.

Construct validity. Operationalizing "Intelligent Agent for SE" and mapping primary studies to our two-level task/technology schema may introduce construct ambiguity. To reduce this threat, we

(i) grounded the working definition in observable capabilities (goal-directed autonomy, perception/-context, planning/reasoning, learning/adaptation, and social interaction), (ii) piloted and iterated the classification rubric on a validation subset before full coding, and (iii) required study-level evidence (tasks, tools, metrics) to justify category assignments. Heterogeneous author terminology (e.g., "assistant" vs. "agent"), overlapping task scopes, and evolving architectural patterns can still blur boundaries; we therefore report representative examples and mark uncertain assignments in the released classification table.

Conclusion validity. The corpus exhibits methodological and metric heterogeneity (e.g., function-level accuracy vs. repository-scale pass rates; costs/latency/memory reported inconsistently). We therefore refrain from effect-size meta-analysis and rely on descriptive aggregation and thematic synthesis, consistent with recommendations for diverse SE evidence bases [73, 128]. This limits causal claims and increases the risk of over-interpretation. To mitigate, we (i) triangulate patterns across independent studies when possible, (ii) qualify claims with the breadth and strength of underlying evidence, and (iii) clearly separate empirical observations from forward-looking recommendations.

External validity. Our search covers April 2023–July 2025 across five major sources (IEEE Xplore, ACM Digital Library, Google Scholar, SpringerLink, and arXiv). The time window and venue scope may bias results toward recent, English-language, and publicly documented artefacts, potentially under-representing proprietary industrial deployments and non-English literature. We used arXiv to ensure recency but applied a peer-review requirement at inclusion time (non–peer-reviewed preprints were excluded), as specified in our eligibility criteria; when only a preprint existed, its status was recorded in our artefacts and excluded from quantitative summaries where appropriate. Given rapid changes in models, tools, and benchmarks, our findings characterize the state of practice within the review window rather than long-term guarantees.

Reproducibility and transparency. To support independent verification and longitudinal extension, we release the review protocol, screening decisions, extraction templates, and classification outputs (repository link provided in Section 2). These artefacts improve auditability and enable re-analysis (e.g., with updated inclusion windows), though they cannot eliminate biases inherent to fast-moving topics and variability in primary-study reporting quality.

8 CONCLUSION

This review synthesizes the rapidly evolving landscape of Intelligent Agents for software engineering across 215 peer-reviewed studies published from April 2023 to July 2025, consolidating how agents are conceptualized, architected, and deployed throughout the SE lifecycle. Our analysis distills a unifying technological backbone—*orchestration engines*, *retrieval-augmented grounding*, *transparent chain/tree-of-thought planners*, and *deep tool-chain integration*—that recurs across task families and underpins reported gains, including accuracy improvements and notable productivity uplifts, while explaining why some prototypes transfer more reliably to repository-scale work. At the same time, evidence remains uneven: many studies rely on narrow, short-horizon tasks with limited reporting of cost, latency, memory, and tool-use success, and underexplored areas, including system integration/tooling and legacy modernization, constrain external validity. Building on these findings, we outline a pragmatic agenda centered on six system-level challenges—evaluation and benchmarks; scalability and efficiency; safety and trustworthiness; human–AI collaboration; integration and deployment; and privacy and governance—calling for unified, multi-modal, cost-aware SDLC benchmarks; lightweight, adaptive agent architectures and budgeted planning; and safety-by-design with auditable traces and, where feasible, formal guarantees. Finally, we emphasize human-centered

co-design, continual and transferable learning, and seamless tool-chain orchestration as necessary conditions for dependable, production-grade adoption. The artefacts released with this study (protocol, screening logs, extraction templates, and classification outputs) are intended to support reproducibility, enable longitudinal re-analysis, and accelerate future research on Intelligent Agent–driven software engineering.

REFERENCES

[1] Vaibhav Aggarwal, Ojasv Kamal, Abhinav Japesh, Zhijing Jin, and Bernhard Schölkopf. 2025. Dars: Dynamic action re-sampling to enhance coding agent performance by adaptive tree traversal. *arXiv preprint arXiv:2503.14269* (2025).

[2] Adem Ait, Javier Luis Cánovas Izquierdo, and Jordi Cabot. 2024. Towards modeling human-agentic collaborative workflows: A bpmn extension. *arXiv preprint arXiv:2412.05958* (2024).

[3] Temitope Akinboyewa, Zhenlong Li, Huan Ning, and M Naser Lessani. 2025. GIS copilot: Towards an autonomous GIS agent for spatial analysis. *International Journal of Digital Earth* 18, 1 (2025), 2497489.

[4] Valtteri Ala-Salmi, Zeeshan Rasheed, Abdul Malik Sami, Zheying Zhang, Kai-Kristian Kemell, Jussi Rasku, Shahbaz Siddeeq, Mika Saari, and Pekka Abrahamsson. 2025. Autonomous Legacy Web Application Upgrades Using a Multi-Agent System. *arXiv preprint arXiv:2501.19204* (2025).

[5] Mohannad Alhanahnah and Yazan Boshmaf. 2024. Depsrag: Towards agentic reasoning and planning for software dependency management. *arXiv preprint arXiv:2405.20455* (2024).

[6] Leonhard Applis, Yuntong Zhang, Shanchao Liang, Nan Jiang, Lin Tan, and Abhik Roychoudhury. 2025. Unified Software Engineering agent as AI Software Engineer. *arXiv preprint arXiv:2506.14683* (2025).

[7] Avi Arora, Jinu Jang, and Roshanak Zilouchian Moghaddam. 2025. SetupBench: Assessing Software Engineering Agents' Ability to Bootstrap Development Environments. *arXiv preprint arXiv:2507.09063* (2025).

[8] Daman Arora, Atharv Sonwane, Nalin Wadhwa, Abhav Mehrotra, Saiteja Utpala, Ramakrishna Bairi, Aditya Kanade, and Nagarajan Natarajan. 2024. Masai: Modular architecture for software-engineering ai agents. *arXiv preprint arXiv:2406.11638* (2024).

[9] Nazmus Ashrafi, Salah Bouktif, and Mohammed Mediani. 2025. Enhancing llm code generation: A systematic evaluation of multi-agent collaboration and runtime debugging for improved accuracy, reliability, and latency. *arXiv preprint arXiv:2505.02133* (2025).

[10] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. *arXiv preprint arXiv:2505.20411* (2025).

[11] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134* (2024).

[12] Islem Bouzenia and Michael Pradel. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. *arXiv preprint arXiv:2506.18824* (2025).

[13] Islem Bouzenia and Michael Pradel. 2025. You name it, I run it: An LLM agent to execute tests of arbitrary projects. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1054–1076.

[14] Timothy Bula, Saurabh Pujar, Luca Buratti, Mihaela Bornea, and Avirup Sil. 2025. SeaView: Software engineering agent visual interface for enhanced workflow. *arXiv preprint arXiv:2504.08696* (2025).

[15] Ira Ceka, Saurabh Pujar, Shyam Ramji, Luca Buratti, Gail Kaiser, and Baishakhi Ray. 2025. Understanding Software Engineering Agents Through the Lens of Traceability: An Empirical Study. *arXiv preprint arXiv:2506.08311* (2025).

[16] Ada Chen, Yongjiang Wu, Junyuan Zhang, Jingyu Xiao, Shu Yang, Jen-tse Huang, Kun Wang, Wenxuan Wang, and Shuai Wang. 2025. A Survey on the Safety and Security Threats of Computer-Using Agents: JARVIS or Ultron? *arXiv preprint arXiv:2505.10924* (2025).

[17] Eason Chen, Chenyu Lin, Xinyi Tang, Aprille Xi, Canwen Wang, Jionghao Lin, and Kenneth R Koedinger. 2025. VTutor: An Open-Source SDK for Generative AI-Powered Animated Pedagogical Agents with Multi-Media Output. *arXiv preprint arXiv:2502.04103* (2025).

[18] John Chen and Uri Wilensky. 2023. ChatLogo: A Large Language Model-Driven Hybrid Natural-Programming Language Interface for Agent-based Modeling and Programming. *arXiv preprint arXiv:2308.08102* (2023).

[19] Yinfang Chen, Manish Shetty, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Jonathan Mace, Chetan Bansal, Rujia Wang, and Saravan Rajmohan. 2025. Aiopslab: A holistic framework to evaluate ai agents for enabling autonomous clouds. *arXiv preprint arXiv:2501.06706* (2025).

[20] Zhi Chen and Lingxiao Jiang. 2025. Evaluating software development agents: Patch patterns, code quality, and issue complexity in real-world github scenarios. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 657–668.

[21] Zhi Chen, Wei Ma, and Lingxiao Jiang. 2025. Unveiling Pitfalls: Understanding Why AI-driven Code Agents Fail at GitHub Issue Resolution. *arXiv preprint arXiv:2503.12374* (2025).
[22] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025. Locagent: Graph-guided llm agents for code localization. *arXiv preprint arXiv:2503.09089* (2025).
[23] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic bug reproduction for effective automated program repair at google. *arXiv preprint arXiv:2502.01821* (2025).
[24] Antoine Chevrot, Alexandre Vernotte, Jean-Rémy Falleri, Xavier Blanc, Bruno Legeard, and Aymeric Cretin. 2025. Are Autonomous Web Agents Good Testers? *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 206–228.
[25] De Chezelles, Thibault Le Sellier, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F Xu, Siva Reddy, Quentin Cappart, et al. 2024. The browsergym ecosystem for web agent research. *arXiv preprint arXiv:2412.05467* (2024).
[26] Yizhou Chi, Yizhang Lin, Sirui Hong, Duyi Pan, Yaying Fei, Guanghao Mei, Bangbang Liu, Tianqi Pang, Jacky Kwok, Ceyao Zhang, et al. 2024. Sela: Tree-search enhanced llm agents for automated machine learning. *arXiv preprint arXiv:2410.17238* (2024).
[27] Jina Chun, Qihong Chen, Jiawei Li, and Iftekhar Ahmed. 2025. Is multi-agent debate (mad) the silver bullet? an empirical analysis of mad in code summarization and translation. *arXiv preprint arXiv:2503.12029* (2025).
[28] Harris Cooper. 2015. *Research synthesis and meta-analysis: A step-by-step approach*. Vol. 2. Sage publications.
[29] Victor Dibia, Jingya Chen, Gagan Bansal, Suff Syed, Adam Fourney, Erkang Zhu, Chi Wang, and Saleema Amershi. 2024. Autogen studio: A no-code developer tool for building and debugging multi-agent systems. *arXiv preprint arXiv:2408.15247* (2024).
[30] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. AgentOps: Enabling Observability of LLM Agents. *arXiv preprint arXiv:2411.05285* (2024).
[31] Wei Dong. 2025. The ann arbor architecture for agent-oriented programming. *arXiv preprint arXiv:2502.09903* (2025).
[32] Changyu Du, Sebastian Esser, Stavros Nousias, and André Borrmann. 2024. Text2BIM: Generating building models using a large language model-based multi-agent framework. *arXiv preprint arXiv:2408.08054* (2024).
[33] Zhuoyun Du, Chen Qian, Wei Liu, Zihao Xie, Yifei Wang, Rennai Qiu, Yufan Dang, Weize Chen, Cheng Yang, Ye Tian, et al. 2025. Multi-Agent Collaboration via Cross-Team Orchestration. In *Findings of the Association for Computational Linguistics: ACL 2025*. 10386–10406.
[34] Venkata Sai Aswath Duvvuru, Bohan Zhang, Michael Vierhauser, and Ankit Agrawal. 2025. LLM-Agents Driven Automated Simulation Testing and Analysis of small Uncrewed Aerial Systems. *arXiv preprint arXiv:2501.11864* (2025).
[35] Gang Fan, Xiaoheng Xie, Xunjin Zheng, Yinan Liang, and Peng Di. 2023. Static code analysis in the ai era: An in-depth exploration of the concept, function, and potential of intelligent code analysis agents. *arXiv preprint arXiv:2310.08837* (2023).
[36] Xiang Fei, Xiawu Zheng, and Hao Feng. 2025. MCP-Zero: Proactive Toolchain Construction for LLM Agents from Scratch. *arXiv preprint arXiv:2506.01056* (2025).
[37] Robert Feldt, Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Towards autonomous testing agents via conversational large language models. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1688–1693.
[38] Sidong Feng, Changhao Du, Huaxiao Liu, Qingnan Wang, Zhengwei Lv, Gang Huo, Xu Yang, and Chunyang Chen. 2025. Agent for User: Testing Multi-User Interactive Features in TikTok. *arXiv preprint arXiv:2504.15474* (2025).
[39] Sidong Feng, Changhao Du, Huaxiao Liu, Qingnan Wang, Zhengwei Lv, Mengfei Wang, and Chunyang Chen. 2025. Breaking Single-Tester Limits: Multi-Agent LLMs for Multi-User Feature Testing. *arXiv preprint arXiv:2506.17539* (2025).
[40] Shiwei Feng, Xiangzhe Xu, Xuan Chen, Kaiyuan Zhang, Syed Yusuf Ahmed, Zian Su, Mingwei Zheng, and Xiangyu Zhang. 2025. IntenTest: Stress Testing for Intent Integrity in API-Calling LLM Agents. *arXiv preprint arXiv:2506.07524* (2025).
[41] Shubham Gandhi, Dhruv Shah, Manasi Patwardhan, Lovekesh Vig, and Gautam Shroff. 2025. Researchcodeagent: An llm multi-agent system for automated codification of research methodologies. In *International Workshop on AI for Transportation*. Springer, 3–37.
[42] Florian Geissler, Karsten Roscher, and Mario Trapp. 2024. Concept-guided llm agents for human-ai safety codesign. In *Proceedings of the AAAI Symposium Series*, Vol. 3. 100–104.
[43] Jing Gong, Yanghui Wu, Linxi Liang, Yanlin Wang, Jiachi Chen, Mingwei Liu, and Zibin Zheng. 2024. CoSQA+: Pioneering the Multi-Choice Code Search Benchmark with Test-Driven Agents. *arXiv preprint arXiv:2406.11589* (2024).

[44] Anastasiia Grishina, Vadim Liventsev, Aki Härmä, and Leon Moonen. 2025. Fully autonomous programming using iterative multi-agent debugging with large language models. *ACM Transactions on Evolutionary Learning* 5, 1 (2025), 1–37.

[45] Chengquan Guo, Xun Liu, Chulin Xie, Andy Zhou, Yi Zeng, Zinan Lin, Dawn Song, and Bo Li. 2024. Redcode: Risky code execution and generation benchmark for code agents. *Advances in Neural Information Processing Systems* 37 (2024), 106190–106236.

[46] Jinyao Guo, Chengpeng Wang, Xiangzhe Xu, Zian Su, and Xiangyu Zhang. 2025. Repoaudit: An autonomous llm-agent for repository-level code auditing. *arXiv preprint arXiv:2501.18160* (2025).

[47] Xuehang Guo, Xingyao Wang, Yangyi Chen, Sha Li, Chi Han, Manling Li, and Heng Ji. 2025. Syncmind: Measuring agent out-of-sync recovery in collaborative software engineering. *arXiv preprint arXiv:2502.06994* (2025).

[48] Tanmay Gupta, Luca Weihs, and Aniruddha Kembhavi. 2024. CodeNav: Beyond tool-use to using real-world codebases with LLM agents. *arXiv preprint arXiv:2406.12276* (2024).

[49] Thorsten Händler. 2023. Balancing autonomy and alignment: a multi-dimensional taxonomy for autonomous LLM-powered multi-agent architectures. *arXiv preprint arXiv:2310.03659* (2023).

[50] Amine Ben Hassouna, Hana Chaari, and Ines Belhaj. 2024. Llm-agent-umf: Llm-based agent unified modeling framework for seamless integration of multi active/passive core-agents. *arXiv preprint arXiv:2409.11393* (2024).

[51] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30.

[52] Li Hu, Guoqiang Chen, Xiuwei Shang, Shaoyin Cheng, Benlong Wu, Gangyang Li, Xu Zhu, Weiming Zhang, and Nenghai Yu. 2025. CompileAgent: Automated Real-World Repo-Level Compilation with Tool-Integrated LLM-based Agent System. *arXiv preprint arXiv:2505.04254* (2025).

[53] Yue Hu, Yuzhu Cai, Yaxin Du, Xinyu Zhu, Xiangrui Liu, Zijie Yu, Yuchen Hou, Shuo Tang, and Siheng Chen. 2024. Self-evolving multi-agent collaboration networks for software development. *arXiv preprint arXiv:2410.16946* (2024).

[54] Yaojie Hu, Qiang Zhou, Qihong Chen, Xiaopeng Li, Linbo Liu, Dejiao Zhang, Amit Kachroo, Talha Oz, and Omer Tripp. 2025. Qualityflow: An agentic workflow for program synthesis controlled by llm quality checks. *arXiv preprint arXiv:2501.17167* (2025).

[55] Jiangping Huang, Dongming Jin, Weisong Sun, Yang Liu, and Zhi Jin. 2025. Knowledge-Guided Multi-Agent Framework for Automated Requirements Development: A Vision. *arXiv preprint arXiv:2506.22656* (2025).

[56] Cory Hymel, Sida Peng, Kevin Xu, and Charath Ranganathan. 2024. Improving Performance of Commercially Available AI Products in a Multi-Agent Configuration. *arXiv preprint arXiv:2410.22129* (2024).

[57] Michael Iannelli, Sneha Kuchipudi, and Vera Dvorak. 2024. SLA Management in Reconfigurable Multi-Agent RAG: A Systems Approach to Question Answering. *arXiv preprint arXiv:2412.06832* (2024).

[58] Andrei Cristian Ionescu, Sergey Titov, and Maliheh Izadi. 2025. A Multi-agent Onboarding Assistant based on Large Language Models, Retrieval Augmented Generation, and Chain-of-Thought. *arXiv preprint arXiv:2503.23421* (2025).

[59] Yoichi Ishibashi and Yoshimasa Nishimura. 2024. Self-organized agents: A llm multi-agent framework toward ultra large-scale code generation and optimization. *arXiv preprint arXiv:2404.02183* (2024).

[60] Kush Jain and Claire Le Goues. 2025. TestForge: Feedback-Driven, Agentic Test Suite Generation. *arXiv preprint arXiv:2503.14713* (2025).

[61] Sarthak Jain, Aditya Dora, Ka Seng Sam, and Prabhat Singh. 2024. Llm agents improve semantic code search. *arXiv preprint arXiv:2408.11058* (2024).

[62] Surya Jasper, Minh Luu, Evan Pan, Aakash Tyagi, Michael Quinn, Jiang Hu, and David Kebo Houngninou. 2025. BugGen: A Self-Correcting Multi-Agent LLM Pipeline for Realistic RTL Bug Synthesis. *arXiv preprint arXiv:2506.10501* (2025).

[63] Dongming Jin, Zhi Jin, Xiaohong Chen, and Chunhui Wang. 2024. Mare: Multi-agents collaboration framework for requirements engineering. *arXiv preprint arXiv:2405.03256* (2024).

[64] Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, and Huaming Chen. 2024. From llms to llm-based agents for software engineering: A survey of current, challenges and future. *arXiv preprint arXiv:2408.02479* (2024).

[65] Haolin Jin, Zechao Sun, and Huaming Chen. 2024. Rgd: Multi-llm based agent debugger via refinement and generation guidance. In *2024 IEEE International Conference on Agents (ICA)*. IEEE, 136–141.

[66] Naizhu Jin, Zhong Li, Tian Zhang, and Qingkai Zeng. 2025. GUARD: Dual-Agent based Backdoor Defense on Chain-of-Thought in Neural Code Generation. *arXiv preprint arXiv:2505.21425* (2025).

[67] Yiyang Jin, Kunzhao Xu, Hang Li, Xueting Han, Yanmin Zhou, Cheng Li, and Jing Bai. 2025. ReVeal: Self-Evolving Code Agents via Iterative Generation-Verification. *arXiv preprint arXiv:2506.11442* (2025).

[68] Thomas Joshi, Shayan Chowdhury, and Fatih Uysal. 2025. SWE-Bench-CL: Continual Learning for Coding Agents. *arXiv preprint arXiv:2507.00014* (2025).

[69] Rabimba Karanjai, Sam Blackshear, Lei Xu, and Weidong Shi. 2025. A Multi-Agent Framework for Automated Vulnerability Detection and Repair in Solidity and Move Smart Contracts. *arXiv preprint arXiv:2502.18515* (2025).
[70] Arsham Gholamzadeh Khoee, Yinan Yu, Robert Feldt, Andris Freimanis, Patrick Andersson Rhodin, and Dhasarathy Parthasarathy. 2024. GoNoGo: An Efficient LLM-based Multi-Agent System for Streamlining Automotive Software Release Decision-Making. In *IFIP International Conference on Testing Software and Systems*. Springer, 30–45.
[71] Myeongsoo Kim, Tyler Stennett, Saurabh Sinha, and Alessandro Orso. 2024. A multi-agent approach for rest api testing with semantic graphs and llm-driven inputs. *arXiv preprint arXiv:2411.07098* (2024).
[72] Seoyeon Kim, Yu Su, and Li-C Wang. 2025. IEA-Plugin: An AI Agent Reasoner for Test Data Analytics. *arXiv preprint arXiv:2504.11496* (2025).
[73] Barbara Kitchenham, O Pearl Brereton, David Budgen, Mark Turner, John Bailey, and Stephen Linkman. 2009. Systematic literature reviews in software engineering–a systematic literature review. *Information and software technology* 51, 1 (2009), 7–15.
[74] He Kong, Die Hu, Jingguo Ge, Liangxiong Li, Tong Li, and Bingzhen Wu. 2025. Vulnbot: Autonomous penetration testing for a multi-agent collaborative framework. *arXiv preprint arXiv:2501.13411* (2025).
[75] Thomas Kuntz, Agatha Duzan, Hao Zhao, Francesco Croce, Zico Kolter, Nicolas Flammarion, and Maksym Andriushchenko. 2025. OS-Harm: A Benchmark for Measuring Safety of Computer Use Agents. *arXiv preprint arXiv:2506.14866* (2025).
[76] Stefano Lambiase, Gemma Catolino, Fabio Palomba, and Filomena Ferrucci. 2024. Motivations, Challenges, Best Practices, and Benefits for Bots and Conversational Agents in Software Engineering: A Multivocal Literature Review. *Comput. Surveys* 57, 4 (2024), 1–37.
[77] Cheryl Lee, Chunqiu Steven Xia, Longji Yang, Jen-tse Huang, Zhouruixin Zhu, Lingming Zhang, and Michael R Lyu. 2024. A unified debugging approach via llm-based multi-agent synergy. *arXiv preprint arXiv:2404.17153* (2024).
[78] Fangjian Lei, Jiawen Liu, Shayan Noei, Ying Zou, Derek Truong, and William Alexander. 2025. Enhancing COBOL Code Explanations: A Multi-Agents Approach Using Large Language Models. *arXiv preprint arXiv:2507.02182* (2025).
[79] Shou Li, Andrey Kan, Laurent Callot, Bhavana Bhasker, Muhammad Shihab Rashid, and Timothy B Esler. 2024. Redo: Execution-free runtime error detection for coding agents. *arXiv preprint arXiv:2410.09117* (2024).
[80] Xinzhe Li. 2025. A review of prominent paradigms for llm-based agents: Tool use, planning (including rag), and feedback learning. In *Proceedings of the 31st International Conference on Computational Linguistics*. 9760–9779.
[81] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, et al. 2024. Personal llm agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459* (2024).
[82] Jenny T Liang, Aayush Kumar, Yasharth Bajpai, Sumit Gulwani, Vu Le, Chris Parnin, Arjun Radhakrishna, Ashish Tiwari, Emerson Murphy-Hill, and Guastavo Soares. 2025. TableTalk: Scaffolding Spreadsheet Development with a Language Agent. *arXiv preprint arXiv:2502.09787* (2025).
[83] Feng Lin, Dong Jae Kim, et al. 2024. Soen-101: Code generation by emulating software process models using large language model agents. *arXiv preprint arXiv:2403.15852* (2024).
[84] Leilei Lin, Yumeng Jin, Yingming Zhou, Wenlong Chen, and Chen Qian. 2024. Mao: A framework for process model generation with multi-agent orchestration. *arXiv preprint arXiv:2408.01916* (2024).
[85] Leilei Lin, Yingming Zhou, Wenlong Chen, and Chen Qian. 2024. Think-on-process: Dynamic process generation for collaborative development of multi-agent system. *arXiv preprint arXiv:2409.06568* (2024).
[86] Zijie Lin, Yiqing Shen, Qilin Cai, He Sun, Jinrui Zhou, and Mingjun Xiao. 2025. AutoP2C: An LLM-Based Agent Framework for Code Repository Generation from Multimodal Content in Academic Papers. *arXiv preprint arXiv:2504.20115* (2025).
[87] Chenxu Liu, Zhiyu Gu, Guoquan Wu, Ying Zhang, Jun Wei, and Tao Xie. 2025. Temac: Multi-Agent Collaboration for Automated Web GUI Testing. *arXiv preprint arXiv:2506.00520* (2025).
[88] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. 2024. Large language model-based agents for software engineering: A survey. *arXiv preprint arXiv:2409.02977* (2024).
[89] Kaiyuan Liu, Youcheng Pan, Yang Xiang, Daojing He, Jing Li, Yexing Du, and Tianrun Gao. 2025. ProjectEval: A Benchmark for Programming Agents Automated Evaluation on Project-Level Code Generation. arXiv:2503.07010 [cs.SE] https://arxiv.org/abs/2503.07010
[90] Siwei Liu, Jinyuan Fang, Han Zhou, Yingxu Wang, and Zaiqiao Meng. 2025. SEW: Self-Evolving Agentic Workflows for Automated Code Generation. *arXiv preprint arXiv:2505.18646* (2025).
[91] Yuanzhe Liu, Ryan Deng, Tim Kaler, Xuhao Chen, Charles E Leiserson, Yao Ma, and Jie Chen. 2025. Lessons Learned: A Multi-Agent Framework for Code LLMs to Learn and Improve. *arXiv preprint arXiv:2505.23946* (2025).
[92] Yizhou Liu, Pengfei Gao, Xinchen Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. 2024. Marscode agent: Ai-native automated bug fixing. *arXiv preprint arXiv:2409.00899* (2024).

[93] Zhengyao Liu, Yunlong Ma, Jingxuan Xu, Junchen Ai, Xiang Gao, Hailong Sun, and Abhik Roychoudhury. 2025. Agent That Debugs: Dynamic State-Guided Vulnerability Repair. *arXiv preprint arXiv:2504.07634* (2025).
[94] Zihan Liu, Ruinan Zeng, Dongxia Wang, Gengyun Peng, Jingyi Wang, Qiang Liu, Peiyu Liu, and Wenhai Wang. 2024. Agents4plc: Automating closed-loop plc code generation and verification in industrial control systems using llm-based agents. *arXiv preprint arXiv:2410.14209* (2024).
[95] Qinghua Lu, Liming Zhu, Xiwei Xu, Zhenchang Xing, Stefan Harrer, and Jon Whittle. 2024. Towards responsible generative ai: A reference architecture for designing foundation model based agents. In *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 119–126.
[96] Wenqiang Luo, Jacky Wai Keung, Boyang Yang, Jacques Klein, Tegawende F Bissyande, Haoye Tian, and Bach Le. 2025. Unlocking LLM Repair Capabilities in Low-Resource Programming Languages Through Cross-Language Translation and Multi-Agent Refinement. *arXiv preprint arXiv:2503.22512* (2025).
[97] Qianou Ma, Hua Shen, Kenneth Koedinger, and Sherry Tongshuang Wu. 2024. How to teach programming in the ai era? using llms as a teachable agent for debugging. In *International Conference on Artificial Intelligence in Education*. Springer, 265–279.
[98] Wei Ma, Daoyuan Wu, Yuqiang Sun, Tianwen Wang, Shangqing Liu, Jian Zhang, Yue Xue, and Yang Liu. 2024. Combining fine-tuning and llm-based agents for intuitive smart contract auditing with justifications. *arXiv preprint arXiv:2403.16073* (2024).
[99] Yingwei Ma, Yongbin Li, Yihong Dong, Xue Jiang, Rongyu Cao, Jue Chen, Fei Huang, and Binhua Li. 2025. Thinking longer, not larger: Enhancing software engineering agents via scaling test-time compute. *arXiv preprint arXiv:2503.23803* (2025).
[100] Yacine Majdoub, Eya Ben Charrada, and Haifa Touati. 2025. Towards Adaptive Software Agents for Debugging. *arXiv preprint arXiv:2504.18316* (2025).
[101] Sun Maojun, Ruijian Han, Binyan Jiang, Houduo Qi, Defeng Sun, Yancheng Yuan, and Jian Huang. 2025. Lambda: A large model based data agent. *J. Amer. Statist. Assoc.* just-accepted (2025), 1–20.
[102] Matias Martinez and Xavier Franch. 2025. Dissecting the SWE-Bench Leaderboards: Profiling Submitters and Architectures of LLM-and Agent-Based Repair Systems. *arXiv preprint arXiv:2506.17208* (2025).
[103] Alex Mathai, Chenxi Huang, Suwei Ma, Jihwan Kim, Hailie Mitchell, Aleksandr Nogikh, Petros Maniatis, Franjo Ivančić, Junfeng Yang, and Baishakhi Ray. 2025. CrashFixer: A crash resolution agent for the Linux kernel. *arXiv preprint arXiv:2504.20412* (2025).
[104] Lukasz Mazur, Nenad Petrovic, James Pontes Miranda, Ansgar Radermacher, Robert Rasche, and Alois Knoll. 2025. Querying Large Automotive Software Models: Agentic vs. Direct LLM Approaches. *arXiv preprint arXiv:2506.13171* (2025).
[105] Glaucia Melo, Paulo Alencar, and Donald Cowan. 2025. Enhancing Software Development with Context-Aware Conversational Agents: A User Study on Developer Interactions with Chatbots. *arXiv preprint arXiv:2505.08648* (2025).
[106] Lingkai Meng, Yu Shao, Long Yuan, Longbin Lai, Peng Cheng, Wenyuan Yu, Wenjie Zhang, Xuemin Lin, and Jingren Zhou. 2025. OSS-UAgent: An Agent-based Usability Evaluation Framework for Open Source Software. *arXiv preprint arXiv:2505.23239* (2025).
[107] Xiangxin Meng, Zexiong Ma, Pengfei Gao, and Chao Peng. 2024. An empirical study on llm-based agents for automated bug fixing. *arXiv preprint arXiv:2411.10213* (2024).
[108] Zhendong Mi, Renming Zheng, Haowen Zhong, Yue Sun, Seth Kneeland, Sayan Moitra, Ken Kutzer, and Zhaozhuo Xu Shaoyi Huang. 2024. CoopetitiveV: Leveraging LLM-powered Coopetitive Multi-Agent Prompting for High-quality Verilog Generation. *arXiv preprint arXiv:2412.11014* (2024).
[109] Ivan Milev, Mislav Balunović, Maximilian Baader, and Martin Vechev. 2025. ToolFuzz–Automated Agent Tool Testing. *arXiv preprint arXiv:2503.04479* (2025).
[110] Louis Milliken, Sungmin Kang, and Shin Yoo. 2025. Beyond pip install: Evaluating llm agents for the automated installation of python projects. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 1–11.
[111] David Moher, Alessandro Liberati, Jennifer Tetzlaff, Douglas G Altman, Prisma Group, et al. 2010. Preferred reporting items for systematic reviews and meta-analyses: the PRISMA statement. *International journal of surgery* 8, 5 (2010), 336–341.
[112] Niels Mündler, Mark Müller, Jingxuan He, and Martin Vechev. 2024. SWT-bench: Testing and validating real-world bug-fixes with code agents. *Advances in Neural Information Processing Systems* 37 (2024), 81857–81887.
[113] Jatin Nainani, Chia-Tung Ho, Anirudh Dhurka, and Haoxing Ren. 2025. Timing Analysis Agent: Autonomous Multi-Corner Multi-Mode (MCMM) Timing Debugging with Timing Debug Relation Graph. *arXiv preprint arXiv:2504.11502* (2025).

[114] Duc SH Nguyen, Bach G Truong, Phuong T Nguyen, Juri Di Rocco, and Davide Di Ruscio. 2025. Teamwork makes the dream work: LLMs-Based Agents for GitHub README. MD Summarization. *arXiv preprint arXiv:2503.10876* (2025).
[115] Anh Nguyen-Duc, Chien Vu Manh, Bao Anh Tran, Viet Phuong Ngo, Luan Le Chi, and Anh Quang Nguyen. 2025. An Empirical Study of Multi-Agent RAG for Real-World University Admissions Counseling. *arXiv preprint arXiv:2507.11272* (2025).
[116] Kaiwen Ning, Jiachi Chen, Jingwen Zhang, Wei Li, Zexu Wang, Yuming Feng, Weizhe Zhang, and Zibin Zheng. 2024. Defining and Detecting the Defects of the Large Language Model-based Autonomous Agents. *arXiv preprint arXiv:2412.18371* (2024).
[117] Robbe Nooyens, Tolgahan Bardakci, Mutlu Beyazit, and Serge Demeyer. 2025. Test Amplification for REST APIs via Single and Multi-Agent LLM Systems. *arXiv preprint arXiv:2504.08113* (2025).
[118] Ana Nunez, Nafis Tanveer Islam, Sumit Kumar Jha, and Peyman Najafirad. 2024. Autosafecoder: A multi-agent framework for securing llm code generation through static analysis and fuzz testing. *arXiv preprint arXiv:2409.10737* (2024).
[119] Marc Oriol, Quim Motger, Jordi Marco, and Xavier Franch. 2025. Multi-Agent Debate Strategies to Enhance Requirements Engineering with Large Language Models. *arXiv preprint arXiv:2507.05981* (2025).
[120] Joshua Owotogbe. 2025. Assessing and Enhancing the Robustness of LLM-based Multi-Agent Systems Through Chaos Engineering. In *2025 IEEE/ACM 4th International Conference on AI Engineering–Software Engineering for AI (CAIN)*. IEEE, 250–252.
[121] Anvith Pabba, Alex Mathai, Anindya Chakraborty, and Baishakhi Ray. 2025. SemAgent: A Semantics Aware Program Repair Agent. *arXiv preprint arXiv:2506.16650* (2025).
[122] Ruwei Pan, Hongyu Zhang, Zhonghao Jiang, and Ran Hou. 2025. AgentDroid: A Multi-Agent Framework for Detecting Fraudulent Android Applications. *arXiv preprint arXiv:2503.12163* (2025).
[123] Ruwei Pan, Hongyu Zhang, and Chao Liu. 2025. CodeCoR: An LLM-based self-reflective multi-agent framework for code generation. *arXiv preprint arXiv:2501.07811* (2025).
[124] Xin Pang, Zhucong Li, Jiaxiang Chen, Yuan Cheng, Yinghui Xu, and Yuan Qi. 2024. Ai2apps: A visual ide for building llm-based ai agent applications. *arXiv preprint arXiv:2404.04902* (2024).
[125] Ghasem Pasandi, Kishor Kunal, Varun Tej, Kunjal Shan, Hanfei Sun, Sumit Jain, Chunhui Li, Chenhui Deng, Teodor-Dumitru Ene, Haoxing Ren, et al. 2025. JARVIS: A Multi-Agent Code Assistant for High-Quality EDA Script Generation. *arXiv preprint arXiv:2505.14978* (2025).
[126] Jirat Pasuksmit, Wannita Takerngsaksiri, Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Ruixiong Zhang, Shiyan Wang, Fan Jiang, Jing Li, Evan Cook, Kun Chen, et al. 2025. Human-In-The-Loop Software Development Agents: Challenges and Future Directions. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 756–757.
[127] Changhua Pei, Zexin Wang, Fengrui Liu, Zeyan Li, Yang Liu, Xiao He, Rong Kang, Tieying Zhang, Jianjun Chen, Jianhui Li, et al. 2025. Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis. In *Companion Proceedings of the ACM on Web Conference 2025*. 422–431.
[128] Mark Petticrew and Helen Roberts. 2008. *Systematic reviews in the social sciences: A practical guide*. John Wiley & Sons.
[129] Huy Nhat Phan, Tien N Nguyen, Phong X Nguyen, and Nghi DQ Bui. 2024. Hyperagent: Generalist software engineering agents to solve coding tasks at scale. *arXiv preprint arXiv:2409.16299* (2024).
[130] Chen Qian, Yufan Dang, Jiahao Li, Wei Liu, Zihao Xie, Yifei Wang, Weize Chen, Cheng Yang, Xin Cong, Xiaoyin Che, et al. 2023. Experiential co-learning of software-developing agents. *arXiv preprint arXiv:2312.17025* (2023).
[131] Chen Qian, Jiahao Li, Yufan Dang, Wei Liu, YiFei Wang, Zihao Xie, Weize Chen, Cheng Yang, Yingli Zhang, Zhiyuan Liu, et al. 2024. Iterative experience refinement of software-developing agents. *arXiv preprint arXiv:2405.04219* (2024).
[132] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. 2023. Chatdev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924* (2023).
[133] Rennai Qiu, Chen Qian, Ran Li, Yufan Dang, Weize Chen, Cheng Yang, Yingli Zhang, Ye Tian, Xuantang Xiong, Lei Han, et al. 2025. Co-Saving: Resource Aware Multi-Agent Collaboration for Software Development. *arXiv preprint arXiv:2505.21898* (2025).
[134] Md Nakhla Rafi, Dong Jae Kim, Tse-Hsun Chen, and Shaowei Wang. 2024. A Multi-Agent Approach to Fault Localization via Graph-Based Retrieval and Reflexion. *arXiv preprint arXiv:2409.13642* (2024).
[135] Alfin Wijaya Rahardja, Junwei Liu, Weitong Chen, Zhenpeng Chen, and Yiling Lou. 2025. Can Agents Fix Agent Issues? *arXiv preprint arXiv:2505.20749* (2025).
[136] Asif Rahman, Veljko Cvetkovic, Kathleen Reece, Aidan Walters, Yasir Hassan, Aneesh Tummeti, Bryan Torres, Denise Cooney, Margaret Ellis, and Dimitrios S. Nikolopoulos. 2025. MARCO: Multi-Agent Code Optimization with Real-Time Knowledge Integration for High-Performance Computing. arXiv:2505.03906 [cs.DC] https://arxiv.org/abs/2505.03906

[137] Zeeshan Rasheed, Malik Abdul Sami, Kai-Kristian Kemell, Muhammad Waseem, Mika Saari, Kari Systä, and Pekka Abrahamsson. 2024. Codepori: Large-scale system for autonomous software development using multi-agent technology. *arXiv preprint arXiv:2402.01411* (2024).

[138] Zeeshan Rasheed, Muhammad Waseem, Aakash Ahmad, Kai-Kristian Kemell, Wang Xiaofeng, Anh Nguyen Duc, and Pekka Abrahamsson. 2024. Can large language models serve as data analysts? a multi-agent assisted approach for qualitative data analysis. *arXiv preprint arXiv:2402.01386* (2024).

[139] Zeeshan Rasheed, Muhammad Waseem, Malik Abdul Sami, Kai-Kristian Kemell, Aakash Ahmad, Anh Nguyen Duc, Kari Systä, and Pekka Abrahamsson. 2024. Autonomous agents in software development: A vision paper. In *International Conference on Agile Software Development*. Springer Nature Switzerland Cham, 15–23.

[140] Zeeshan Rasheed, Muhammad Waseem, Kari Systä, and Pekka Abrahamsson. 2024. Large language model evaluation via multi ai agents: Preliminary results. *arXiv preprint arXiv:2404.01023* (2024).

[141] Muhammad Shihab Rashid, Christian Bock, Yuan Zhuang, Alexander Buchholz, Tim Esler, Simon Valentin, Luca Franceschi, Martin Wistuba, Prabhu Teja Sivaprasad, Woo Jung Kim, et al. 2025. SWE-PolyBench: A multi-language benchmark for repository level evaluation of coding agents. *arXiv preprint arXiv:2504.08703* (2025).

[142] Benjamin Rombaut, Sogol Masoumzadeh, Kirill Vasilevski, Dayi Lin, and Ahmed E Hassan. 2024. Watson: A Cognitive Observability Framework for the Reasoning of LLM-Powered Agents. *arXiv preprint arXiv:2411.03455* (2024).

[143] Francesco Romeo, Luigi Arena, Francesco Blefari, Francesco Aurelio Pironti, Matteo Lupinacci, and Angelo Furfaro. 2025. ARPaCCino: An Agentic-RAG for Policy as Code Compliance. *arXiv preprint arXiv:2507.10584* (2025).

[144] Krishna Ronanki. 2025. Facilitating Trustworthy Human-Agent Collaboration in LLM-based Multi-Agent System oriented Software Engineering. arXiv:2505.04251 [cs.SE] https://arxiv.org/abs/2505.04251

[145] Pat Rondon, Renyao Wei, José Cambronero, Jürgen Cito, Aaron Sun, Siddhant Sanyam, Michele Tufano, and Satish Chandra. 2025. Evaluating agent-based program repair at google. *arXiv preprint arXiv:2501.07531* (2025).

[146] Devjeet Roy, Xuchao Zhang, Rashi Bhave, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. Exploring llm-based agents for root cause analysis. In *Companion proceedings of the 32nd ACM international conference on the foundations of software engineering*. 208–219.

[147] Abhik Roychoudhury, Corina Pasareanu, Michael Pradel, and Baishakhi Ray. 2025. Agentic ai software engineer: Programming with trust. *arXiv preprint arXiv:2502.13767* (2025).

[148] Stuart Russell, Peter Norvig, and Artificial Intelligence. 1995. A modern approach. *Artificial Intelligence. Prentice-Hall, Egnlewood Cliffs* 25, 27 (1995), 79–80.

[149] Amirali Sajadi, Kostadin Damevski, and Preetha Chatterjee. 2025. Are AI-Generated Fixes Secure? Analyzing LLM and Agent Patches on SWE-bench. *arXiv preprint arXiv:2507.02976* (2025).

[150] Abdul Malik Sami, Zeeshan Rasheed, Kai-Kristian Kemell, Muhammad Waseem, Terhi Kilamo, Mika Saari, Anh Nguyen Duc, Kari Systä, and Pekka Abrahamsson. 2024. System for systematic literature review using multiple ai agents: Concept and an empirical evaluation. *arXiv preprint arXiv:2403.08399* (2024).

[151] Humza Sami, Samy Charas, Asav Gandhi, Pierre-Emmanuel Gaillardon, Valerio Tenace, et al. 2025. Nexus: A Lightweight and Scalable Multi-Agent Framework for Complex Tasks Automation. *arXiv preprint arXiv:2502.19091* (2025).

[152] Malik Abdul Sami, Muhammad Waseem, Zeeshan Rasheed, Mika Saari, Kari Systä, and Pekka Abrahamsson. 2024. Experimenting with multi-agent software development: Towards a unified platform. *arXiv preprint arXiv:2406.05381* (2024).

[153] Ranjan Sapkota, Konstantinos I Roumeliotis, and Manoj Karkee. 2025. Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic ai. *arXiv preprint arXiv:2505.19443* (2025).

[154] Anjana Sarkar and Soumyendu Sarkar. 2025. Survey of LLM Agent Communication with MCP: A Software Design Pattern Centric Review. *arXiv preprint arXiv:2506.05364* (2025).

[155] Haiyang Shen, Yue Li, Desong Meng, Dongqi Cai, Sheng Qi, Li Zhang, Mengwei Xu, and Yun Ma. 2024. Shortcutsbench: A large-scale real-world benchmark for API-based agents. *arXiv preprint arXiv:2407.00132* (2024).

[156] Manish Shetty, Yinfang Chen, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Xuchao Zhang, Jonathan Mace, Dax Vandevoorde, Pedro Las-Casas, Shachee Mishra Gupta, et al. 2024. Building ai agents for autonomous clouds: Challenges and design principles. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*. 99–110.

[157] Shahbaz Siddeeq, Zeeshan Rasheed, Malik Abdul Sami, Mahade Hasan, Muhammad Waseem, Jussi Rasku, Mika Saari, Kai-Kristian Kemell, and Pekka Abrahamsson. 2025. Distributed Approach to Haskell Based Applications Refactoring with LLMs Based Multi-Agent Systems. *arXiv preprint arXiv:2502.07928* (2025).

[158] Shahbaz Siddeeq, Muhammad Waseem, Zeeshan Rasheed, Md Mahade Hasan, Jussi Rasku, Mika Saari, Henri Terho, Kalle Makela, Kai-Kristian Kemell, and Pekka Abrahamsson. 2025. LLM-based Multi-Agent System for Intelligent Refactoring of Haskell Code. *arXiv preprint arXiv:2506.19481* (2025).

[159] HoHyun Sim, Hyeonjoong Cho, Yeonghyeon Go, Zhoulai Fu, Ali Shokri, and Binoy Ravindran. 2025. Large Language Model-Powered Agent for C to Rust Code Translation. *arXiv preprint arXiv:2505.15858* (2025).

[160] Ramneet Singh, Sathvik Joel, Abhav Mehrotra, Nalin Wadhwa, Ramakrishna B Bairi, Aditya Kanade, and Nagarajan Natarajan. 2025. Code Researcher: Deep Research Agent for Large Systems Code and Commit History. *arXiv preprint arXiv:2506.11060* (2025).
[161] Wannita Takerngsaksiri, Jirat Pasuksmit, Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Ruixiong Zhang, Fan Jiang, Jing Li, Evan Cook, Kun Chen, and Ming Wu. 2024. Human-in-the-loop software development agents. *arXiv preprint arXiv:2411.12924* (2024).
[162] Xunzhu Tang, Kisub Kim, Yewei Song, Cedric Lothritz, Bei Li, Saad Ezzini, Haoye Tian, Jacques Klein, and Tegawendé F Bissyandé. 2024. Codeagent: Autonomous communicative agents for code review. *arXiv preprint arXiv:2402.02172* (2024).
[163] Yun Tang, Mengbang Zou, Zeinab Nezami, Syed Ali Raza Zaidi, and Weisi Guo. 2025. KP-A: A Unified Network Knowledge Plane for Catalyzing Agentic Network Intelligence. *arXiv preprint arXiv:2507.08164* (2025).
[164] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. 2024. Magis: Llm-based multi-agent framework for github issue resolution. *Advances in Neural Information Processing Systems* 37 (2024), 51963–51993.
[165] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. *arXiv preprint arXiv:2407.18901* (2024).
[166] Fernando Vallecillos Ruiz. 2024. Agent-driven automatic software improvement. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 470–475.
[167] Mandana Vaziri, Louis Mandel, Yuji Watanabe, Hirokuni Kitahara, Martin Hirzel, and Anca Sailer. 2025. Representing Prompting Patterns with PDL: Compliance Agent Case Study. *arXiv preprint arXiv:2507.06396* (2025).
[168] Bowei Wang, Qi Xiong, Zeqing Xiang, Lei Wang, and Renzhi Chen. 2025. Rtlsquad: Multi-agent based interpretable rtl design. *arXiv preprint arXiv:2501.05470* (2025).
[169] Jialin Wang and Zhihua Duan. 2025. Empirical Research on Utilizing LLM-based Agents for Automated Bug Fixing via LangGraph. *arXiv preprint arXiv:2502.18465* (2025).
[170] Kaixin Wang, Tianlin Li, Xiaoyu Zhang, Chong Wang, Weisong Sun, Yang Liu, and Bin Shi. 2025. Software Development Life Cycle Perspective: A Survey of Benchmarks for Code Large Language Models and Agents. *arXiv preprint arXiv:2505.05283* (2025).
[171] Taiming Wang, Yanjie Jiang, Chunhao Dong, Yuxia Zhang, and Hui Liu. 2025. Context-Aware Code Wiring Recommendation with LLM-based Agent. *arXiv preprint arXiv:2507.01315* (2025).
[172] Xinchen Wang, Pengfei Gao, Chao Peng, Ruida Hu, and Cuiyun Gao. 2025. CodeVisionary: An Agent-based Framework for Evaluating Large Language Models in Code Generation. *arXiv preprint arXiv:2504.13472* (2025).
[173] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2024. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741* (2024).
[174] Yanlin Wang, Wanjun Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. 2024. Agents in software engineering: Survey, landscape, and vision. *arXiv preprint arXiv:2409.09030* (2024).
[175] Zehao Wang, Dong Jae Kim, and Tse-Hsun Chen. 2024. Identifying performance-sensitive configurations in software systems through code analysis with llm agents. *arXiv preprint arXiv:2406.12806* (2024).
[176] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Jihong Wang, Fengbin Yin, Lunting Fan, Lingfei Wu, and Qingsong Wen. 2024. Rcagent: Cloud root cause analysis by autonomous agents with tool-augmented large language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 4966–4974.
[177] Yunze Wei, Xiaohui Xie, Yiwei Zuo, Tianshuo Hu, Xinyi Chen, Kaiwen Chi, and Yong Cui. 2025. Leveraging LLM Agents for Translating Network Configurations. *arXiv preprint arXiv:2501.08760* (2025).
[178] Xin-Cheng Wen, Jiaxin Ye, Cuiyun Gao, Lianwei Wu, and Qing Liao. 2024. EvalSVA: Multi-Agent Evaluators for Next-Gen Software Vulnerability Assessment. *arXiv preprint arXiv:2501.14737* (2024).
[179] Jules White. 2024. Building living software systems with generative & agentic AI. *arXiv preprint arXiv:2408.01768* (2024).
[180] Michael Wooldridge. 2009. *An introduction to multiagent systems*. John wiley & sons.
[181] Jie JW Wu and Fatemeh H Fard. 2024. Humanevalcomm: Benchmarking the communication competence of code generation for llms and llm agent. *arXiv preprint arXiv:2406.00215* (2024).
[182] Boming Xia, Qinghua Lu, Liming Zhu, Zhenchang Xing, Dehai Zhao, and Hao Zhang. 2024. Evaluation-Driven Development of LLM Agents: A Process Model and Reference Architecture. *arXiv preprint arXiv:2411.13768* (2024).
[183] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* (2024).

[184] Xiao Xia, Dan Zhang, Zibo Liao, Zhenyu Hou, Tianrui Sun, Jing Li, Ling Fu, and Yuxiao Dong. 2024. Scenegenagent: Precise industrial scene generation with coding agent. *arXiv preprint arXiv:2410.21909* (2024).
[185] Yuchen Xia, Manthan Shenoy, Nasser Jazdi, and Michael Weyrich. 2023. Towards autonomous system: flexible modular production system enhanced with large language model agents. In *2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 1–8.
[186] Yanzheng Xiang, Hanqi Yan, Shuyin Ouyang, Lin Gui, and Yulan He. 2025. Scireplicate-bench: Benchmarking llms in agent-driven algorithmic reproduction from research papers. *arXiv preprint arXiv:2504.00255* (2025).
[187] Yijia Xiao, Runhui Wang, Luyang Kong, Davor Golac, and Wei Wang. 2025. CSR-Bench: Benchmarking LLM agents in deployment of computer science research repositories. *arXiv preprint arXiv:2502.06111* (2025).
[188] Mingzhe Xing, Rongkai Zhang, Hui Xue, Qi Chen, Fan Yang, and Zhen Xiao. 2024. Understanding the weakness of large language model agents within a complex android environment. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6061–6072.
[189] Mengwei Xu. 2025. Every software as an agent: Blueprint and case study. *arXiv preprint arXiv:2502.04747* (2025).
[190] Qinghua Xu, Guancheng Wang, Lionel C Briand, and Kui Liu. 2025. Hallucination to Consensus: Multi-Agent LLMs for End-to-End Test Generation with Accurate Oracles. *CoRR abs/2506.02943* (2025).
[191] Yisen Xu, Feng Lin, Jinqiu Yang, Nikolaos Tsantalis, et al. 2025. Mantra: Enhancing automated method-level refactoring with contextual rag and multi-agent llm collaboration. *arXiv preprint arXiv:2503.14340* (2025).
[192] Haoran Xue, Gias Uddin, and Song Wang. 2025. PAGENT: Learning to Patch Software Engineering Agents. *arXiv preprint arXiv:2506.17772* (2025).
[193] Shuo Yan, Ruochen Li, Ziming Luo, Zimu Wang, Daoyang Li, Liqiang Jing, Kaiyu He, Peilin Wu, George Michalopoulos, Yue Zhang, et al. 2025. LMR-BENCH: Evaluating LLM Agent's Ability on Reproducing Language Modeling Research. *arXiv preprint arXiv:2506.17335* (2025).
[194] Dayu Yang, Antoine Simoulin, Xin Qian, Xiaoyi Liu, Yuwei Cao, Zhaopu Teng, and Grey Yang. 2025. DocAgent: A Multi-Agent System for Automated Code Documentation Generation. *arXiv preprint arXiv:2504.08725* (2025).
[195] Weiqing Yang, Hanbin Wang, Zhenghao Liu, Xinze Li, Yukun Yan, Shuo Wang, Yu Gu, Minghe Yu, Zhiyuan Liu, and Ge Yu. 2024. Coast: Enhancing the code debugging ability of llms through communicative agent based data synthesis. *arXiv preprint arXiv:2408.05006* (2024).
[196] Eray Yapağcı, Yavuz Alp Sencer Öztürk, and Eray Tüzün. 2025. BugCraft: End-to-End Crash Bug Reproduction Using LLM Agents in Minecraft. *arXiv preprint arXiv:2503.20036* (2025).
[197] Juyeon Yoon, Robert Feldt, and Shin Yoo. 2023. Autonomous large language model agents enabling intent-driven mobile gui testing. *arXiv preprint arXiv:2311.08649* (2023).
[198] Boxi Yu, Yuxuan Zhu, Pinjia He, and Daniel Kang. 2025. Utboost: Rigorous evaluation of coding agents on swe-bench. *arXiv preprint arXiv:2506.09289* (2025).
[199] Fenglin Yu, Fangkai Yang, Xiaoting Qin, Zhiyang Zhang, Jue Zhang, Qingwei Lin, Hongyu Zhang, Yingnong Dang, Saravan Rajmohan, Dongmei Zhang, et al. 2025. Enabling Autonomic Microservice Management through Self-Learning Agents. *arXiv preprint arXiv:2501.19056* (2025).
[200] Jiapeng Yu, Yuqian Wu, Yajing Zhan, Wenhao Guo, Zhou Xu, and Raymond Lee. 2024. Co-learning: code learning for multi-agent reinforcement collaborative framework with conversational natural language interfaces. *arXiv preprint arXiv:2409.00985* (2024).
[201] Zhongming Yu, Hejia Zhang, Yujie Zhao, Hanxian Huang, Matrix Yao, Ke Ding, and Jishen Zhao. 2025. Orcaloca: An llm agent framework for software issue localization. *arXiv preprint arXiv:2502.00350* (2025).
[202] Zhengmin Yu, Yuan Zhang, Ming Wen, Yinan Nie, Wenhui Zhang, and Min Yang. 2025. CXXCrafter: An LLM-Based Agent for Automated C/C++ Open Source Software Building. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2618–2640.
[203] Zhiqiang Yuan, Weitong Chen, Hanlin Wang, Kai Yu, Xin Peng, and Yiling Lou. 2024. Transagent: An llm-based multi-agent system for code translation. *arXiv preprint arXiv:2409.19894* (2024).
[204] Karina Zainullina, Alexander Golubev, Maria Trofimova, Sergei Polezhaev, Ibragim Badertdinov, Daria Litvintseva, Simon Karasik, Filipp Fisin, Sergei Skvortsov, Maksim Nekrashevich, et al. 2025. Guided Search Strategies in Non-Serializable Environments with Applications to Software Engineering Agents. *arXiv preprint arXiv:2505.13652* (2025).
[205] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. 2024. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762* (2024).
[206] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. *arXiv preprint arXiv:2401.07339* (2024).
[207] Kexun Zhang, Weiran Yao, Zuxin Liu, Yihao Feng, Zhiwei Liu, Rithesh Murthy, Tian Lan, Lei Li, Renze Lou, Jiacheng Xu, et al. 2024. Diversity empowers intelligence: Integrating expertise of software engineering agents. *arXiv preprint arXiv:2408.07060* (2024).

[208] Ke Zhang, Chenxi Zhang, Chong Wang, Chi Zhang, YaChen Wu, Zhenchang Xing, Yang Liu, Qingshan Li, and Xin Peng. 2025. LogiAgent: Automated Logical Testing for REST Systems with LLM-Based Multi-Agents. *arXiv preprint arXiv:2503.15079* (2025).

[209] Lingzhe Zhang, Yunpeng Zhai, Tong Jia, Xiaosong Huang, Chiming Duan, and Ying Li. 2025. AgentFM: Role-Aware Failure Management for Distributed Databases with LLM-Driven Multi-Agents. *arXiv preprint arXiv:2504.06614* (2025).

[210] Tao Zhang, Zhenhai Liu, Yong Xin, and Yongjun Jiao. 2025. MooseAgent: A LLM Based Multi-agent Framework for Automating Moose Simulation. *arXiv preprint arXiv:2504.08621* (2025).

[211] Xuanming Zhang, Yuxuan Chen, Yuan Yuan, and Minlie Huang. 2025. Towards Exception Safety Code Generation with Intermediate Representation Agents Framework. arXiv:2410.06949 [cs.SE] https://arxiv.org/abs/2410.06949

[212] Xuanming Zhang, Yuxuan Chen, Yiming Zheng, Zhexin Zhang, Yuan Yuan, and Minlie Huang. 2024. Seeker: Towards exception safety code generation with intermediate language agents framework. *arXiv preprint arXiv:2412.11713* (2024).

[213] Yiran Zhang, Ruiyin Li, Peng Liang, Weisong Sun, and Yang Liu. 2025. Knowledge-Based Multi-Agent Framework for Automated Software Architecture Design. *arXiv preprint arXiv:2503.20536* (2025).

[214] Yaolun Zhang, Yinxu Pan, Yudong Wang, and Jie Cai. 2024. Pybench: Evaluating llm agent on various real-world coding tasks. *arXiv preprint arXiv:2407.16732* (2024).

[215] Zheying Zhang, Maruf Rayhan, Tomas Herda, Manuel Goisauf, and Pekka Abrahamsson. 2024. Llm-based agents for automating the enhancement of user story quality: An early report. In *International Conference on Agile Software Development*. Springer Nature Switzerland Cham, 117–126.

[216] Zixiao Zhao, Jing Sun, Zhe Hou, Zhiyuan Wei, Cheng-Hao Cai, Miao Qiao, and Jin Song Dong. 2024. MaCTG: Multi-Agent Collaborative Thought Graph for Automatic Programming. *arXiv preprint arXiv:2410.19245* (2024).

[217] Mingwei Zheng, Chengpeng Wang, Xuwei Liu, Jinyao Guo, Shiwei Feng, and Xiangyu Zhang. 2025. An LLM Agent for Functional Bug Detection in Network Protocols. *arXiv preprint arXiv:2506.00714* (2025).

[218] Zhenhao Zhou, Zhuochen Huang, Yike He, Chong Wang, Jiajun Wang, Yijian Wu, Xin Peng, and Yiling Lou. 2025. Benchmarking and Enhancing LLM Agents in Localizing Linux Kernel Bugs. *arXiv preprint arXiv:2505.19489* (2025).

[219] Andrew Zhu, Liam Dugan, and Chris Callison-Burch. 2024. ReDel: A Toolkit for LLM-Powered Recursive Multi-Agent Systems. *arXiv preprint arXiv:2408.02248* (2024).

[220] Ming Zhu and Yi Zhou. 2024. MOSS: Enabling Code-Driven Evolution and Context Management for AI Agents. *arXiv preprint arXiv:2409.16120* (2024).

[221] Yueheng Zhu, Chao Liu, Xuan He, Xiaoxue Ren, Zhongxin Liu, Ruwei Pan, and Hongyu Zhang. 2025. AdaCoder: An Adaptive Planning and Multi-Agent Framework for Function-Level Code Generation. *arXiv preprint arXiv:2504.04220* (2025).

[222] Xinkai Zou, Yan Liu, Xiongbo Shi, and Chen Yang. 2025. Goal2Story: A Multi-Agent Fleet based on Privately Enabled sLLMs for Impacting Mapping on Requirements Elicitation. *arXiv preprint arXiv:2503.13279* (2025).